

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Прикладной математики
(полное название кафедры)

УТВЕРЖДАЮ

Зав. кафедрой

Соловейчик Ю.Г.
(фамилия, имя, отчество)

(подпись)

« _____ » _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Михайлина Александра Александровича

.....
(фамилия, имя, отчество студента – автора работы)

Разработка программной системы

с использованием движка Unity

для создания карточной игры в жанре "Roguelike"

.....
(тема работы)

Факультет Прикладной математики и информатики

(полное название факультета)

Направление подготовки 01.03.02. Прикладная математика и информатика

(код и наименование направления подготовки бакалавра)

**Руководитель
от НГТУ**

Задорожный А.Г.
(фамилия, имя, отчество)

К.Т.Н., доцент
(ученая степень, ученое звание)

.....
(подпись, дата)

**Автор выпускной
квалификационной работы**

Михайлин А.А.
(фамилия, И.О.)

ФПМИ, ПМ-13
(факультет, группа)

.....
(подпись, дата)

Новосибирск, 2025 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Прикладной математики
(полное название кафедры)

УТВЕРЖДАЮ

Зав. кафедрой

Соловейчик Ю.Г.

(фамилия, имя, отчество)

«___» _____ 2025 г.

(подпись)

**ЗАДАНИЕ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ БАКАЛАВРА**

студенту Михайлину Александру Александровичу
(фамилия, имя, отчество студента)

Направление подготовки 01.03.02. Прикладная математика и информатика

Факультет Прикладной математики и информатики

Тема Разработка программной системы с использованием движка Unity

для создания карточной игры в жанре "Roguelike"

Исходные данные (или цель работы):

Разработать программную систему, включающую в себя функциональные
компоненты для создания карточной игры в жанре "Roguelike" с использованием
игрового движка Unity. В рамках поставленной цели предусматривается реализация
основных игровых механик, таких как процедурная генерация, карточный бой, система
событий, а также сохранение игрового прогресса, с обеспечением модульной
архитектуры, пригодной для последующего расширения и развития проекта.

Структурные части работы:

1. Изучение теоретических основ и жанровых особенностей

2. Изучение и применение технических средств разработки на движке Unity.

3. Разработка ключевых игровых компонентов.

4. Анализ гибкости проекта и его возможностей для расширения.

Задание согласовано и принято к исполнению.

**Руководитель
от НГТУ**

Задорожный А.Г.

(фамилия, имя, отчество)

к.т.н., доцент

(ученая степень, ученое звание)

(подпись, дата)

Студент

Михайлин А.А.

(фамилия, имя, отчество)

ФПМИ, ПМ-13

(факультет, группа)

(подпись, дата)

Тема утверждена приказом по НГТУ № 1457/2 от « 14 » 03 2025 г.

ВКР сдана в ГЭК №____, тема сверена с данными приказа

(подпись секретаря экзаменационной комиссии по защите ВКР, дата)

(фамилия, имя, отчество секретаря экзаменационной комиссии по защите ВКР)

АННОТАЦИЯ

Отчёт 93 стр., 5 ч., 14 рис., 10 источников, 4 прил.

РАЗРАБОТКА ИГРЫ, РАЗРАБОТКА СКРИПТОВ, ДВИЖОК UNITY, ПРОЦЕДУРНАЯ ГЕНЕРАЦИЯ, СОХРАНЕНИЕ ДАННЫХ, ЗАЩИТА ДАННЫХ

Цель работы – разработка программной системы с использованием движка Unity для создания карточной игры в жанре "Roguelike"

В результате была разработана система для разработки игр в жанре «Roguelike» с использованием игрового движка Unity. Проект сочетает в себе трёхмерную навигацию по подземелью и двухмерные пошаговые бои с использованием карточной системы. Основное внимание было уделено архитектуре процедурной генерации, реализации событий с последствиями, гибкой боевой системе и сохранению данных с защитой от изменений.

СОДЕРЖАНИЕ

Введение.....	6
1. Обзор жанра и использованных в разработке технических средств	8
1.1. Особенности жанра карточных рогаиков.....	8
1.2. Влияние случайности и процедурной генерации на геймплей. Примеры игр и анализ основных механик	9
1.3. Причины использования Unity. Компоненты и технологии, применённые в проекте.....	12
2. Реализация пользовательского интерфейса	15
2.1. Дизайн UX для карточного рогаика.....	15
2.2. Разработка и реализация главного меню игры	16
2.3. Карта подземелья	20
2.4. Интерфейс боя	22
2.5. Используемые изображения.....	24
3. Игровой процесс	26
3.1. Пошаговый бой, основные правила	26
3.2. Алгоритмы генерации комнат и коридоров	28
3.3. Интерактивные элементы подземелья	32
3.4. Типы событий и их особенности.....	35
4. Система хранения данных.....	38
4.1. Хранение основной информации о забеге.....	38
4.2. Хранение информации о персонажах	39
4.3. Хранение информации о навыках	41
4.4. Хранение информации о событиях	42
4.5. Формат сериализации данных	45
4.6. Защита файлов сохранений от изменений.....	46
5. Потенциал повторного использования	51
5.1. Гибкость боевой системы и расширяемость навыков.....	51
5.2. Универсальность системы событий.....	53
5.3. Возможности адаптации проекта под другие игровые жанры.....	56

Заключение	58
Список литературы	59
Приложение А. Репозиторий	60
Приложение Б. Файл DungeonGenerator.cs.....	61
Приложение В. Файл Fight.cs.....	68
Приложение Г. Файл Fighter.cs.....	84

ВВЕДЕНИЕ

Компьютерные игры остаются одним из самых востребованных видов цифрового досуга, объединяя в себе технологии, визуальное искусство и интерактивный дизайн. Особое место среди них занимает жанр карточных рогаляков, сочетающий элементы случайной генерации, стратегического планирования и повторного прохождения с уникальными условиями. В такой игре обеспечение логики мира важно не меньше, чем его визуальная сторона: игрок должен ощущать, что его выбор действительно влияет на дальнейшие события.

В процессе изучения жанровых особенностей и поиска эффективных решений для реализации ключевых механик я обращался не только к академической литературе, но и к практическим онлайн-ресурсам. Одним из таких ресурсов стало руководство «*Let's Build a Roguelike*» [9], ориентированное на язык `FreeBasic`. Несмотря на это, оно оказалось полезным при проработке логики и структуры игры. В нём последовательно разбираются этапы создания рогаляка, включая генерацию уровней, систему боя и взаимодействие с окружением. Я использовал это руководство как основу для понимания базовых принципов жанра и почерпнул из него идеи по построению модульного кода, организации данных и внедрению случайных событий, адаптируя предложенные методики под реализацию в Unity и на языке C#.

В процессе реализации я изучал документацию Unity [4] и активно руководствовался ею при разработке ключевых компонентов. Руководство позволило глубже разобраться в устройстве игрового движка, работе с пользовательским интерфейсом, системами сериализации и обработки ввода. Благодаря этому удалось заложить устойчивую техническую основу проекта и понять внутреннюю логику Unity как среды для создания игр.

При проектировании игровых механик и пользовательского опыта я учитывал подход Джесси Шелла, изложенный в книге «*Геймдизайн. Как создать игру, в которую будут играть все*» [2]. Основной мыслью автора является то, что

успешная игра должна быть выстроена вокруг четких психологических ожиданий игрока и включать в себя элементы мотивации, интереса и вовлечения. Особое внимание Шелл уделяет важности выбора, последствий и эмоционального отклика, что напрямую повлияло на моё стремление создавать события и ситуации, в которых решения игрока имеют значимый результат, усиливая эффект погружения и осмысленности происходящего.

Особое внимание уделялось нарративному дизайну: для передачи тёмной, морально нагруженной атмосферы я использовал приёмы из руководства по «тяжёлому» тексту Rayne Hall [10], добавляя дихотомию выбора и психологические дилеммы в каждое событие. В результате строится не просто цепочка комнат и боёв, а наполненный смыслом мир, где каждое столкновение с врагом несёт эмоциональную нагрузку – так, чтобы игрок не просто «играл», а проживал историю.

1. ОБЗОР ЖАНРА И ИСПОЛЬЗОВАННЫХ В РАЗРАБОТКЕ ТЕХНИЧЕСКИХ СРЕДСТВ

1.1. ОСОБЕННОСТИ ЖАНРА КАРТОЧНЫХ РОГАЛИКОВ

Жанр карточных рогаликов представляет собой гибрид двух популярных направлений в геймдизайне: карточных стратегий и roguelike-игр. Он сочетает в себе элементы построения колоды, случайной генерации контента, постоянного риска и высокого уровня реиграбельности. Такие игры часто строятся вокруг коротких, но насыщенных сессий, где каждое прохождение отличается от предыдущего, а успех зависит как от стратегических решений игрока, так и от случайных факторов.

Рогалик (Roguelike) – это жанр компьютерных игр, берущий своё начало с классической *Rogue* 1980 года выхода (рисунок 1). Он характеризуется процедурной генерацией уровней, необратимой смертью персонажа (перманентной потерей прогресса) и высокой сложностью. Игрок каждый раз начинает заново, сталкиваясь с новыми условиями и испытаниями, что делает каждое прохождение уникальным.

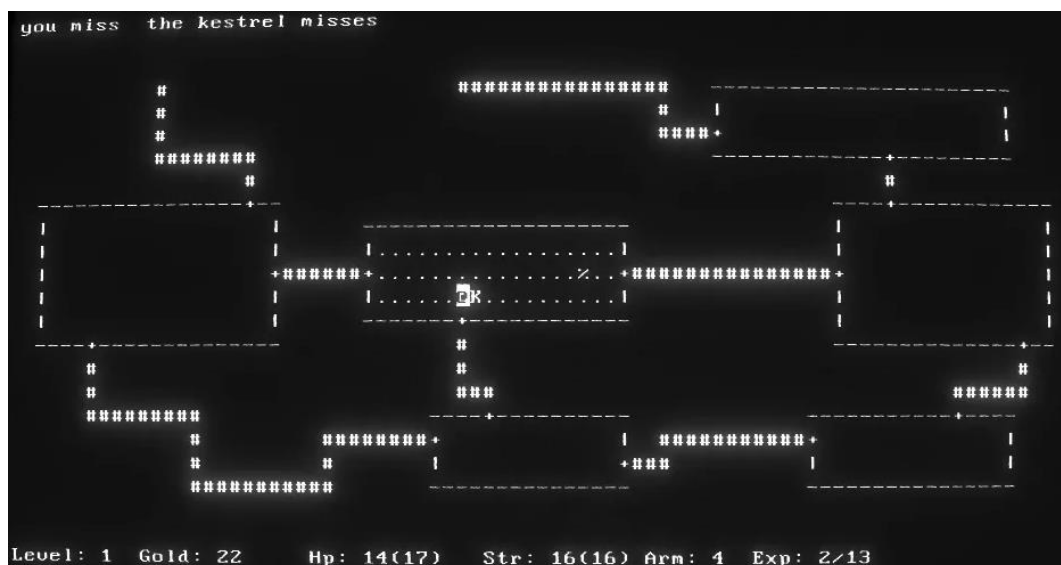


Рисунок 1 – Игра *Rogue*

Карточные стратегии используют карты как основной элемент игрового процесса. Каждая карта представляет собой определённое действие, эффект, заклинание или юнита. Игроки строят колоду и применяют её в бою или других игровых ситуациях, разыгрывая карты в нужный момент и адаптируясь к изменениям.

Объединяя ключевые черты этих жанров, карточные рогаики создают глубокий и вариативный игровой опыт. От roguelike они перенимают случайную генерацию событий, локаций и наград, а также элемент "перманентной смерти", заставляющий переосмысливать стратегию в каждом новом забеге. От карточных стратегий – механику построения колоды, где игрок постепенно собирает карты в течение прохождения, усиливая синергию между ними и формируя уникальный стиль игры. Такое сочетание делает жанр особенно привлекательным для тех, кто ценит и тактику, и постоянные вызовы.

Одной из ключевых особенностей жанра является постоянное принятие решений – выбор пути, события, новой карты для своей колоды. Награды или потери всегда несут последствия, которые сложно предсказать заранее.

Кроме того, карточные рогаики обычно обладают высокой степенью процедурной генерации – уровни, события, награды и даже противники формируются случайным образом, что делает каждую партию уникальной. При этом важным остаётся баланс между случайностью и контролем – игрок должен ощущать, что его решения значимы, несмотря на влияние случайных факторов.

1.2. ВЛИЯНИЕ СЛУЧАЙНОСТИ И ПРОЦЕДУРНОЙ ГЕНЕРАЦИИ НА ГЕЙМПЛЕЙ.

ПРИМЕРЫ ИГР И АНАЛИЗ ОСНОВНЫХ МЕХАНИК

Случайность и процедурная генерация играют ключевую роль в структуре карточных рогаиков, напрямую влияя на геймплей, баланс и реиграбельность. Эти элементы обеспечивают непредсказуемость событий, создавая уникальные

сценарии в каждом прохождении и заставляя игрока адаптироваться к меняющимся условиям.

Процедурная генерация, как правило, применяется при создании уровней. Она позволяет увеличить интерес к игре и добавить разнообразия между сессиями. Но остаётся важным соблюдать баланс между случайными и статичными моментами: слишком хаотичный набор препятствий или наград может разрушить положительный опыт взаимодействия с игрой, в то время как предсказуемость убирает элемент реиграбельности.

В карточных рогалях случайность проявляется даже при сборе колоды. Игрок не получает заранее выбранные карты, а строит колоду по мере продвижения. При этом задействуется гибкость мышления и стратегическая импровизация. Также уменьшается вероятность собирать одни и те же карты, ради повторения выгодной колоды раз за разом. Вместо того чтобы полагаться на заранее составленные стратегии, игрок использует имеющиеся ресурсы ради поиска наиболее выгодной стратегии.

Одним из наиболее известных представителей жанра является Slay the Spire – игра, которая заложила основу современной формулы: последовательное прохождение комнат, боёв и событий с колодой карт, которую игрок постепенно собирает в течение одного забега. Важной чертой игры является то, что каждая карта – это отдельное действие, а успех в бою напрямую зависит от продуманности и синергии между картами.

Однако одной только случайности недостаточно для создания увлекательного игрового процесса. Важно понимать, что удачная реализация случайных событий требует контроля и баланса. Большинство современных игр в жанре используют техники взвешенной генерации (weighted random), контекстные ограничения, системы компенсации за неудачи и другие механизмы, направленные на снижение ощущения «несправедливой» случайности. Например, в вышеупомянутой Slay the Spire после нескольких слабых предложений игроку может быть гарантирована карта более высокого качества. Это создаёт ощущение честности

и позволяет игроку чувствовать влияние своих решений, несмотря на присутствие случайных факторов.

Ещё одним ярким примером можно назвать *Monster Train*, развивающий идеи *Slay the Spire* в другом направлении. В нём добавлены многослойные поля боя (несколько этажей), фракции с уникальной механикой и комбинирование двух колод, что повышает вариативность стратегии. Эта игра демонстрирует, как можно использовать знакомые элементы жанра, привнося при этом оригинальные концепции, усиливающие геймплей.

Игра *Vault of the Void* делает акцент на контроле над колодой, позволяя игроку гибко удалять и заменять карты даже в бою. Это уменьшает влияние случайности и поощряет планирование. Подобный подход показывает, как внутри жанра возможны смещения акцентов – от рогалика с сильной случайностью к более управляемой стратегии.

Анализируя эти проекты, можно выделить несколько общих и отличительных механик:

- **Постепенное формирование колоды:** практически во всех играх игрок начинает с базового набора карт и расширяет его, получая награды.
- **Случайность и процедурность:** генерируются как комнаты и враги, так и выбор наград, что делает каждое прохождение уникальным.
- **Синергия между картами:** успех игрока зависит не столько от силы отдельных карт, сколько от их взаимодействия между собой.
- **Наличие событий с выбором:** многие игры дополняются нелинейными ситуациями, где решения влияют на ход игры.

В рамках данной работы реализована собственная вариация жанра, сочетающая пошаговые карточные сражения с исследованием процедурно сгенерированного подземелья. Игроку предлагается самостоятельно выбирать направление движения между комнатами, исследуя лабиринт, в котором расположение событий и врагов случайно. Это повышает ощущение погружения и делает каждое прохождение уникальным. В большинстве комнат игрок сталкивается с

событиями, в которых необходимо сделать выбор – варианты могут привести к получению награды, изменению характеристик или даже повышению скрытого уровня "тёмной кармы", что отражает моральные последствия действий. Некоторые события можно пропустить, но есть и обязательные для прохождения. Боевая система основана на классической пошаговой механике боя: игрок выбирает действия доступных персонажей, разыгрывая навыки в виде карт. Такой подход делает бой более прозрачным и фокусирует внимание на тактике и выборе умений, а не на управлении ресурсами.

1.3. ПРИЧИНЫ ИСПОЛЬЗОВАНИЯ UNITY. КОМПОНЕНТЫ И ТЕХНОЛОГИИ, ПРИМЕНЁННЫЕ В ПРОЕКТЕ

Выбор игрового движка является одним из ключевых этапов в начале разработки любого проекта. В случае разработки карточного рогалика с элементами процедурной генерации и пошаговых сражений, оптимальным решением стал игровой движок Unity. Его широкие возможности, поддержка 2D и 3D-графики, а также развитая экосистема делают его универсальным инструментом для реализации проектов различной сложности.

Одной из главных причин выбора Unity стала поддержка гибридных решений: в рассматриваемом проекте используется 3D-навигация по подземелью и 2D-боевая система. Unity позволяет без труда сочетать оба режима в рамках одной сцены или проекта. Это позволяет сильно упростить и ускорить разработку.

Unity обладает удобной системой UI, которая особенно важна для карточных игр, ведь они имеют большое количество визуальных и интерактивных элементов. Благодаря встроенной системе Canvas и поддержке анимаций, возможно быстро и гибко настраивать пользовательский интерфейс, адаптируя его под нужды проекта.

Unity позволяет реализовать очень разнообразные системы. В нём можно создавать уникальные игровые системы, такие как генерации подземелий,

сохранения, обработки событий, поведения врагов и т.д. Наличие большого количества готовых решений и документации, а также активное сообщество разработчиков позволяют быстро находить ответы на возникающие вопросы.

Для реализации проекта были использованы ключевые компоненты Unity, которые обеспечили необходимую функциональность и удобство разработки. Прежде всего, были задействованы `GameObjects`. Они являются основой для структурирования игровой сцены и управления различными элементами, такими как персонажи, окружение и интерфейс. С помощью его компонентов, таких как `Transform`, можно точно задавать положение, вращение и масштаб. Эти возможности использовались для правильного расположения элементов на игровом поле.

Для хранения характеристик существ, заклинаний, событий и настроек генерации подземелья были использованы `Scriptable Object`. Они позволяют удобно работать с данными и хранить их как файлы вне сцены. Это позволило упростить управление игровыми данными и повысить гибкость проекта.

Префабы (`Prefabs`) были использованы в большей степени для построения подземелья. Префабы позволили создать шаблоны объектов, которые можно многократно использовать для создания случайных уровней.

Для обеспечения физики объектов в 3D-пространстве был использован компонент `Rigidbody` и коллайдеры. Они необходимы для создания реалистичных взаимодействий в игровых сценах, таких как столкновения объектов или взаимодействие со сценой.

Система лучей – `Raycast` была использована для реализации наведения на объекты в мире и кнопки на `World Space Canvas` (UI интерфейс, расположенный не на экране игрока, а в пространстве). Это позволило точно определить, на какие элементы интерфейса наведен указатель мыши. Использовались 2 типа лучей, один тип для физических лучей, а второй для UI элементов.

Как было сказано ранее, в проекте использована 3D и 2D-графика в рамках единого игрового цикла. Исследование подземелья реализовано от первого лица: игрок перемещается по пространству, состоящему из комнат и соединяющих их

коридоров. Это позволило создать атмосферу погружения, ориентированную на исследование. Камера, управление и интерфейс адаптированы под этот режим, включая элементы вроде мини-карты и всплывающих панелей с характеристиками персонажа.

При начале боя происходит плавный переход к сцене сражения, реализованной в 2D-плоскости. Здесь камера фиксируется сбоку, а интерфейс перестраивается под пошаговый бой: отображаются карты навыков и шкалы здоровья. Такой подход позволяет сохранить визуальную читаемость и традиционную структуру карточных боёв, свойственную жанру.

С технической точки зрения, сцены 3D и 2D обрабатываются отдельно, но используют общие данные о состоянии игрока, противниках, инвентаре и текущем прогрессе. Благодаря этому обеспечена согласованность между режимами, а переходы между ними происходят быстро и без потери контекста.

2. РЕАЛИЗАЦИЯ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

2.1. ДИЗАЙН UX ДЛЯ КАРТОЧНОГО РОГАЛИКА

Жанр карточных рогаляков требует особого внимания к пользовательскому интерфейсу (UI) и опыту взаимодействия с игрой (UX), поскольку геймплей во многом строится на принятии тактических решений, навигации по событиям, управлении колодой и постоянной работе с информацией. Даже при глубокой и интересной игровой механике плохой интерфейс может свести на нет весь пользовательский интерес.

Ключевые особенности UI/UX для карточного рогалика:

1. **Чёткость и наглядность информации.** Игроку важно моментально понимать текущую игровую ситуацию: какие карты доступны, сколько урона они наносят, сколько ресурсов осталось, какие эффекты активны на врагах и союзниках. Для этого используются понятные иконки, всплывающие подсказки, цветовые акценты и визуальные шкалы (например, здоровья или энергии).
2. **Простота взаимодействия с картами.** Важным элементом UX является интуитивная система перетаскивания карт, наведения курсора для просмотра подробной информации, а также чёткая обратная связь на каждое действие (подсветка возможных целей, анимации выбора и применения карт).
3. **Масштабируемость интерфейса под разные разрешения.** Поскольку игра разрабатывается в Unity с использованием Canvas и компонентов RectTransform, интерфейс легко адаптируется под различные разрешения и пропорции экрана. Это особенно важно для РС и портативных платформ.
4. **Отдельные режимы отображения для боёв и исследования.** В карточных рогаляках часто используется смена интерфейса в зависимости от контекста: в исследовании подземелья – карта локаций и краткая информация о событиях, в бою – карточная панель, индикаторы врагов, эффекты статусов.

Это переключение интерфейсов помогает не перегружать экран и делает игровой процесс более читабельным.

5. Минимализм против перегрузки. Поскольку жанр рогаляков уже предполагает постоянную генерацию новых ситуаций и изменение условий, UX должен помогать игроку быстро адаптироваться. Чрезмерное количество кнопок, окон и неструктурированной информации может вызвать у игрока усталость и путаницу. Поэтому важным принципом является визуальная иерархия: самое важное – на виду, остальное доступно при наведении или клике.

6. Влияние на геймплей. Хорошо продуманный интерфейс повышает скорость принятия решений, снижает нагрузку на память игрока и позволяет ему больше сосредоточиться на тактической составляющей, а не на борьбе с системой управления. UX-дизайн напрямую влияет на реиграбельность: чем удобнее взаимодействовать с игрой, тем легче завлечь игрока в новый забег.

2.2. РАЗРАБОТКА И РЕАЛИЗАЦИЯ ГЛАВНОГО МЕНЮ ИГРЫ

Главное меню представляет собой первую точку взаимодействия игрока с игрой, задавая её тон и формируя общее впечатление. Поэтому при разработке главного меню особое внимание уделялось как визуальному оформлению, так и удобству взаимодействия с элементами интерфейса.

Главное меню оформлено в виде статичного фонового изображения, поверх которого располагаются пять текстовых кнопок, размещённых по вертикальной дуге с изгибом вправо. В нижнем правом углу планируется разместить название игры, органично вписанное в композицию экрана. Текущая версия главного меню представлена на рисунке 2.

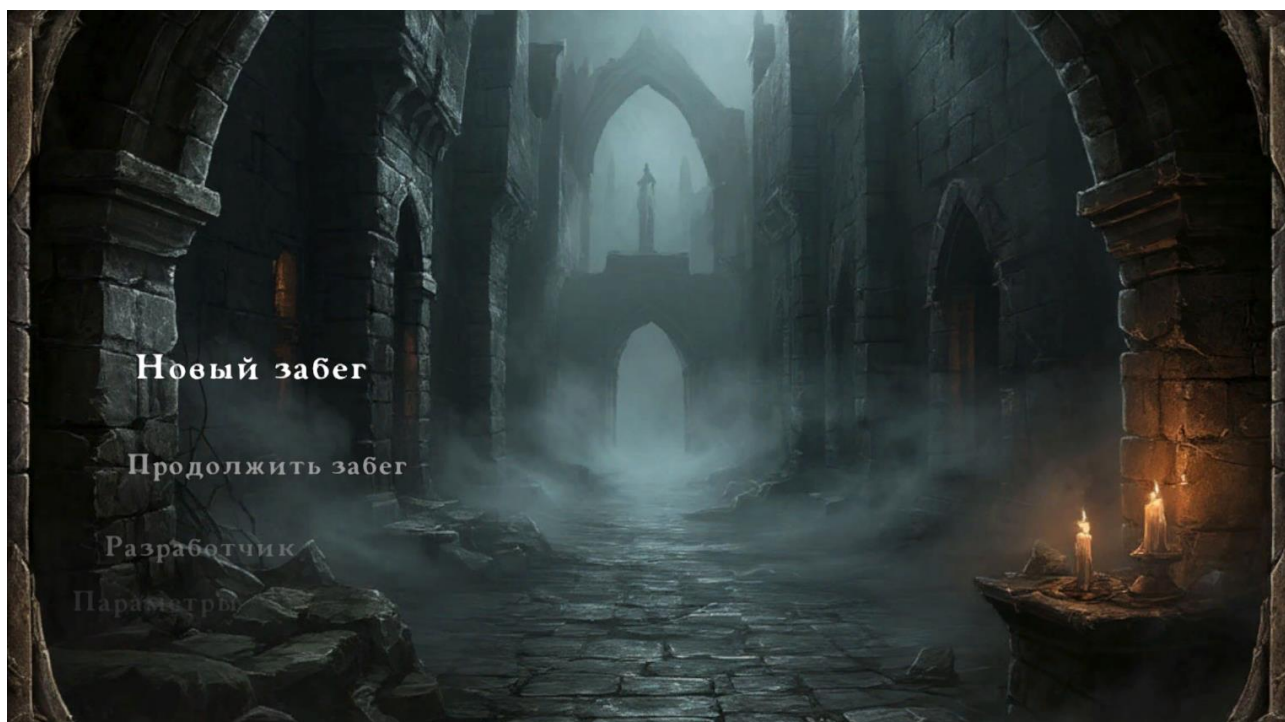


Рисунок 2 – Главное меню игры

Список доступных пунктов меню:

- Новый забег;
- Продолжить забег;
- Разработчик;
- Параметры;
- Выход.

Интерактивные кнопки реализованы с акцентом на текущую, выделенную пользователем. Такая кнопка:

- Увеличена по размеру относительно остальных.
- Является более насыщенной по цвету.
- Все остальные кнопки приглушены и полупрозрачны.

Перемещение между пунктами меню реализовано несколькими способами, чтобы пользователь мог использовать самый удобный для себя:

- Кнопками стрелка вверх/вниз.

- Колёсиком мыши.
- Кликом по нужной кнопке.

Выбор активного пункта осуществляется:

- Enter (срабатывает активный пункт).
- Кликом на уже выделенный элемент.

При выборе одного из пунктов меню, справа, поверх основного фона плавно появляется соответствующее окно. Анимация реализована через изменение прозрачности – от 0 до 1. Каждому пункту меню соответствует своё окно. Пример такого окна представлен на рисунке 3.

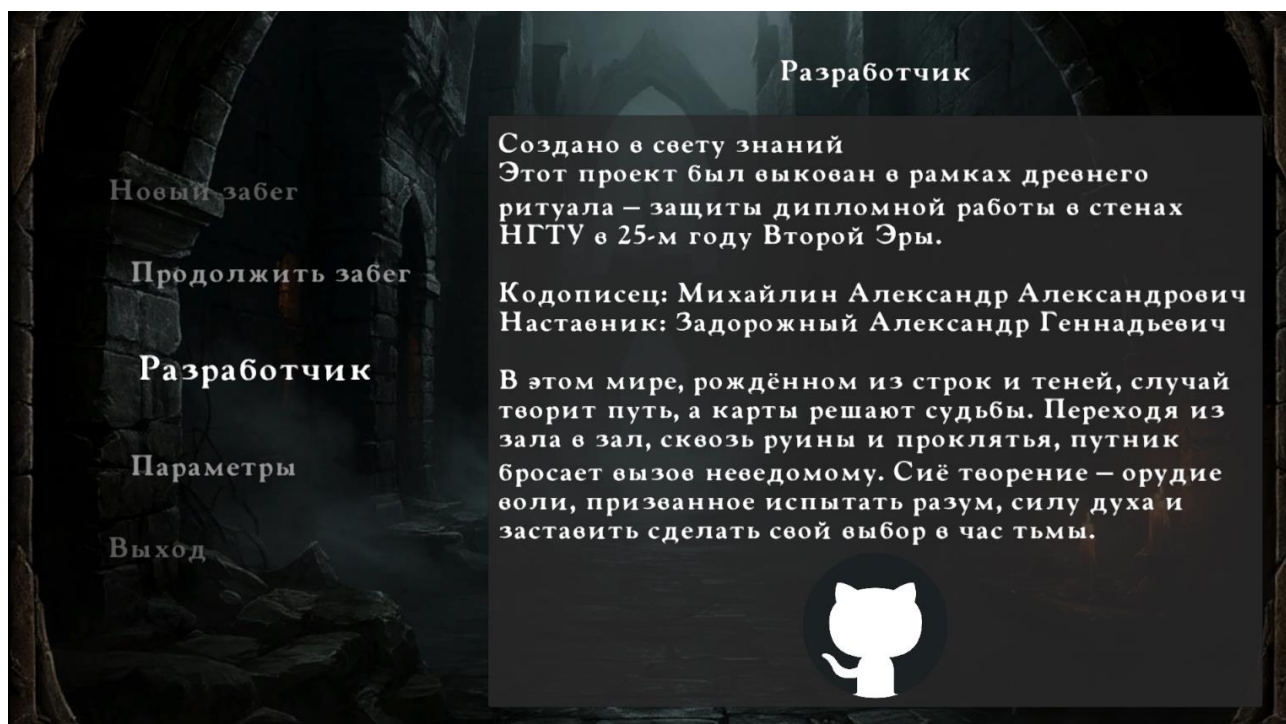


Рисунок 3 – Окно «Разработчик»

Особенности отдельных пунктов:

- «Выход»: открывает диалог подтверждения выхода из игры с двумя кнопками – «Да» и «Нет»;
- «Разработчик»: описывает проект в стилистике тёмного фэнтези;

- «Параметры»: окно содержит настройки громкости звука и музыки;
- «Продолжить забег»: если в сохранениях отсутствуют данные по всем трём слотам, кнопка становится неактивной и принимает серый цвет;
- «Новый забег» и «Продолжить забег»: открывают окно выбора одного из трёх слотов сохранения. В случае нового забега после выбора слота отображается экран выбора стартового персонажа: Воин, Лучница или Жрица. На экране для каждого класса написано краткое описание.

Окно выбора персонажа можно увидеть на рисунке 4, в проекте при переключении персонажей воспроизводится анимация. Она обеспечивает плавный переход между ними, акцентируя внимание на текущем выборе. При нажатии на центрального персонажа, начинается игра (этим вы подтверждаете свой выбор).

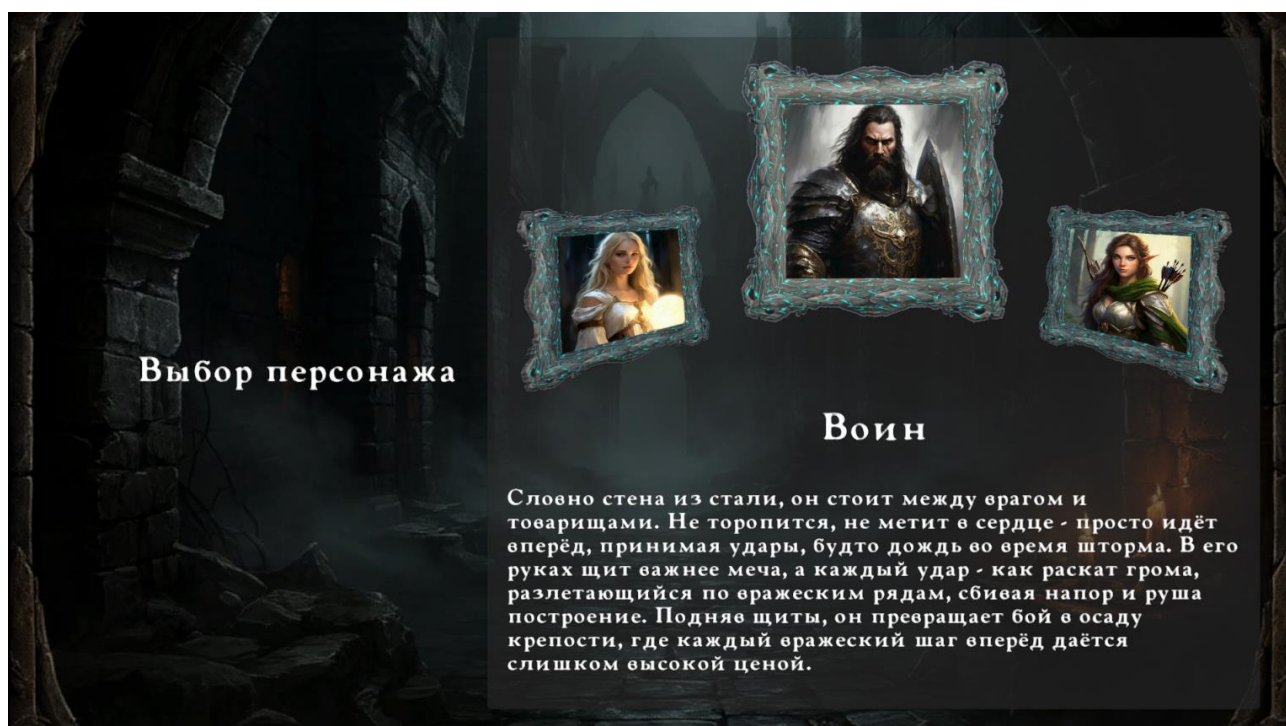


Рисунок 4 – Окно выбора персонажа

2.3. КАРТА ПОДЗЕМЕЛЬЯ

Мини-карта – это небольшое отображение подземелья, расположенное в верхнем правом углу экрана. Она вращается в зависимости от текущего направления взгляда игрока, для облегчения ориентирования в пространстве. Это позволяет игроку всегда знать где он находится и где он ещё не был. Пример мини-карты и её увеличенной версии можно увидеть на рисунке 5.



Рисунок 5 – Мини-карта и её увеличенная версия

Основные особенности мини-карты:

- Отображает текущую локацию игрока и близлежащие комнаты;
- Используется для ориентира, позволяя игроку быстро ориентироваться в подземелье;
- Вращается в зависимости от направления взгляда игрока, чтобы всегда оставаться в правильном положении.

Основные особенности увеличенной версии карты:

- Открывается при нажатии клавиши Tab и занимает весь экран;

- Отображает более подробную информацию о подземелье;
- Как и мини-карта, полная карта также вращается в зависимости от направления взгляда.

Туман войны:

- Используется для сокрытия неизведанных частей подземелья. Система автоматически открывает новые участки карты, когда игрок исследует новые комнаты или коридоры;
- Туман войны можно снять, если у игрока есть определённые глобальные навыки, такие как "След древнего маршрута".

Код разделяется на два класса: `MiniMapUI` и `MiniMapFullUI`.

Класс `MiniMapUI`:

- Основной класс, управляющий отображением мини-карты и полной карты;
- Управляет переключением между мини-картой и полной картой при нажатии клавиши `Tab`;
- Использует методы `ShiftMap()` и `FogOfWar()` для динамического обновления позиции карты и для отображения/скрытия частей карты в зависимости от состояния игры (например, разблокировка или скрытие комнат);
- Отображает комнаты и коридоры подземелья, создавая объекты на основе префабов в зависимости от положения и ориентации каждого объекта.

Класс `MiniMapFullUI`:

- Класс, отвечающий за отображение отдельных элементов карты (комнаты и коридоры);
- Привязывает объекты на мини-карте к конкретным комнатам или коридорам подземелья.

Мини-карта и полная карта синхронизированы, и их элементы всегда обновляются в зависимости от положения игрока в подземелье. Функция `ShiftMap()` отвечает за обновление координат карты, когда игрок перемещается по подземелью, чтобы актуализировать положение объектов. Метод `FogOfWar()` обновляет видимость элементов карты, открывая их по мере того, как игрок продвигается.

2.4. ИНТЕРФЕЙС БОЯ

Пользовательский интерфейс боевой системы играет ключевую роль в карточных рогаляках, напрямую влияя на интуитивность, скорость взаимодействия и общее качество игрового опыта. Реализация боевого интерфейса в проекте опирается на компактность, визуальную структуру и отзывчивость элементов. Пример того, как выглядит боевой интерфейс, представлен на рисунке 6.



Рисунок 6 – Боевой интерфейс

После выбора активного персонажа игроку сразу же отображается горизонтальная панель со всеми доступными навыками в виде карточек. Эти карточки являются центральным элементом интерфейса и выполняют сразу несколько функций:

- **Информативность:** каждая карточка содержит название и иконку, а описание действия открывается при наведении курсора на карточку.
- **Анимационная отзывчивость:** при наведении курсора карточка слегка приподнимается, создавая эффект отклика и подчёркивая готовность к выбору, а также открывается окно с подробной информацией. Пример представлен на рисунке 7.
- **Фиксация выбора:** при нажатии карточка остаётся приподнятой, визуально сигнализируя, что действие выбрано. Выбор другой карточки автоматически сбрасывает предыдущее выделение.
- **Быстрый доступ:** все действия и взаимодействия происходят на одном экране, без перехода между окнами.

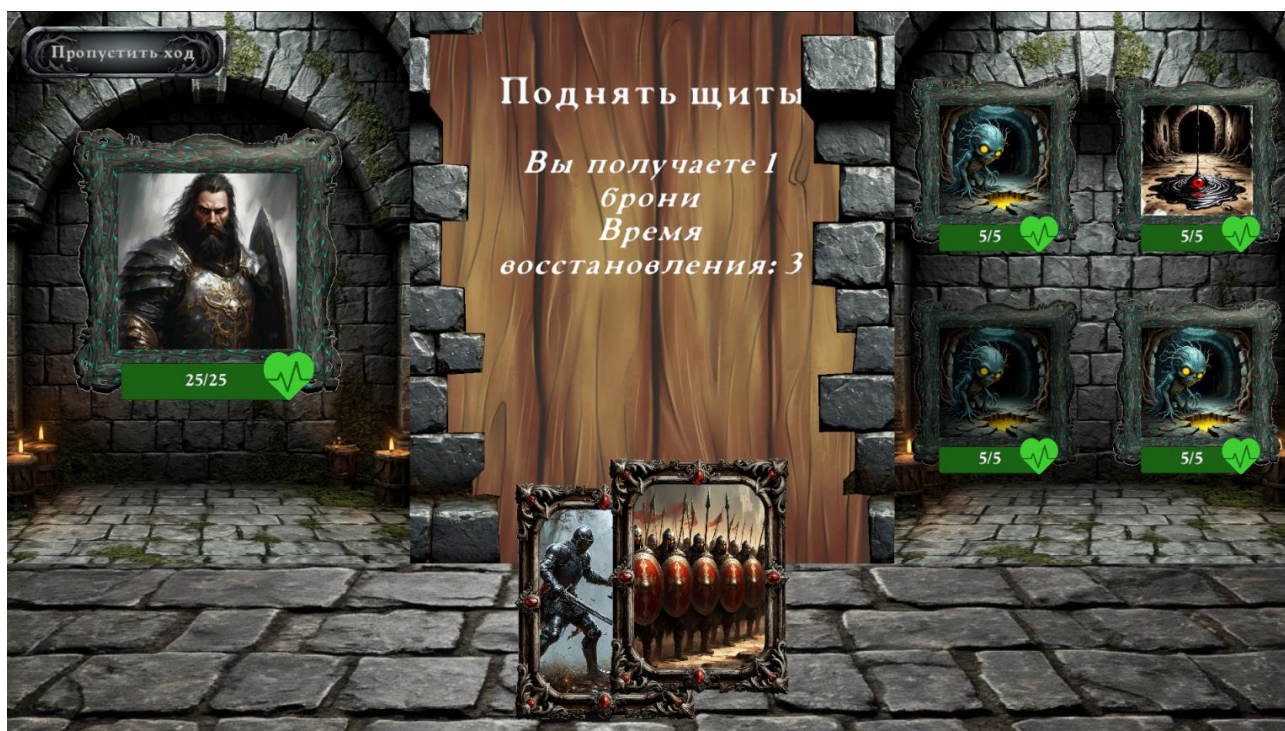


Рисунок 7 – Описание навыка

После выбора действия игрок сразу может указать цель, не покидая текущего экрана. Это значительно сокращает количество кликов и повышает скорость принятия решений. Поддерживается свободное переключение между

навыками, пока действие не подтверждено, что позволяет быстро адаптироваться к изменяющейся ситуации.

Если в момент взаимодействия не выбран ни один навык, игрок может кликнуть по любому персонажу (своему или вражескому), чтобы открыть всплывающее окно со всеми активными на нём пассивными эффектами и статусами. Это позволяет быстро уточнить влияние различных модификаторов без перегрузки основного интерфейса.

Практически любой выбор можно отменить, а окно закрыть нажатием правой кнопки мыши. За исключением применения навыка.

Все элементы интерфейса стилизованы под эстетику карточного рогалика: используются декоративные рамки, акценты на активных элементах и приятные визуальные эффекты при наведении и выборе. Благодаря этому боевой процесс ощущается цельным, быстрым и визуально удовлетворяющим.

2.5. ИСПОЛЬЗОВАННЫЕ ИЗОБРАЖЕНИЯ

Изображения для данного проекта создавались при помощи нейросети «Шедеврум» от компании «Яндекс».

«Шедеврум» – это инструмент от компании Яндекс, который использует технологии искусственного интеллекта для генерации изображений на основе текстовых запросов. Он работает с использованием нейросетей, которые интерпретируют описание пользователя и создают соответствующую визуальную композицию.

Пример запроса для генерации изображения: upper half body, fantasy art, dark fantasy style, woman, priestess of light, white corset, light orb before her, face in focus, little mouth, closed mouth, faint smile, small nose, thin brown eyebrows, brown green eyes, cheerful eyes, blonde hair, bright white light in a blurred background, background with oiled brush, character draw with clear boundaries. Результат этого запроса представлен на рисунке 8. Дополнительные примеры изображений, сгенерированных для проекта, можно увидеть на рисунке 9.



Рисунок 8 – Результат генерации изображения «Шедевром»



Рисунок 9 – Пример генерации изображений, используемых в проекте

3. ИГРОВОЙ ПРОЦЕСС

3.1. ПОШАГОВЫЙ БОЙ, ОСНОВНЫЕ ПРАВИЛА

Боевая система проекта реализована в формате пошагового взаимодействия между командой игрока и противниками. Каждый бой состоит из последовательных раундов, в рамках которых участники поочерёдно совершают действия. Важной особенностью данной системы является гибкость хода и влияние характеристик персонажей на порядок и результат сражения.

В начале каждого раунда определяется порядок ходов. Приоритет предоставляется стороне персонажа с наивысшей характеристикой «Ловкость» среди всех участников боя – как героев, так и врагов. При равенстве наивысшей характеристики у обеих сторон, выбор производится случайно. После этого ходы чередуются, пока каждая сторона не исчерпает свои доступные ходы. В случае, если одна сторона завершает свои действия раньше, оставшиеся ходы полностью переходят к противостоящей стороне.

По завершении всех ходов:

- Применяются эффекты, действующие с течением времени (яд, лечение, разложение и т.д.);
- Обновляется время восстановления умений;
- Персонажи готовятся к новому раунду: очищаются списки походивших, обновляется визуальное представление команд, иницируется расчет намерений врагов.

Для отображения активных участников боя используется специальная система представления команд:

- Левая сторона экрана отображает игрока, его отряд, выбранного героя и цели врагов;
- Правая сторона содержит врагов или цели, выбранные героем при активации навыков;

- Смена отображения команд происходит в зависимости от текущего активного участника и выбранной цели.

Боевая система основана на использовании карт, которые делятся на два типа – активные и пассивные. Они формируют индивидуальную колоду персонажа и определяют его возможности в бою.

Активные карты – это действия, которые игрок может использовать в свой ход. Они включают в себя: атаки, защитные манёвры, навыки поддержки и в целом всё что угодно. Обычно карта требует выбора цели на поле боя и имеет время восстановления в один или несколько ходов.

Пассивные карты – это эффекты, которые активируются автоматически при выполнении определённых условий или действуют постоянно. Они обеспечивают различные бонусы, такие как повышение характеристик, автоматический ответный урон, защита от конкретных эффектов или усиление других карт. Эти карты делают стиль боя персонажа уникальным.

Карточная система тесно связана с характеристиками персонажей. Эффективность карт часто напрямую зависит от параметров героя, таких как Сила, Ловкость или Мудрость. Это стимулирует игроков подбирать колоды, соответствующие боевому стилю выбранного героя, а также экспериментировать с различными комбинациями активных и пассивных навыков.

Характеристика «Сила» напрямую определяет урон от обычной атаки. Чем выше значение этой характеристики у героя, тем больший базовый урон она способна нанести. Кроме того, Сила участвует в механике запугивания врагов, которая активируется в начале каждого раунда.

В начале раунда игроку предоставляется возможность запугать врагов. Для каждого противника производится индивидуальный расчет вероятности испуга, зависящий от разницы значений Силы между атакующим героем и целью. При успешном запугивании враг полностью теряет возможность совершить действия в текущем раунде, что может значительно повлиять на ход боя.

Характеристика «Ловкость» влияет на очередность ходов в бою. Персонаж с наивысшей ловкостью получает право начать первый ход в раунде.

Дополнительно, при высоком значении ловкости, герой может получить дополнительный (двойной) ход, дающий ему стратегическое преимущество.

Характеристика «Мудрость» определяет способность героя видеть запланированные действия врагов (намерения). Чем выше значение мудрости, тем выше шанс, что персонаж увидит намерения. Информацию о них можно увидеть, наведя курсор на портрет врага.

Каждая характеристика играет уникальную роль:

- Сила – отвечает за урон обычной атаки и контроль поля боя;
- Ловкость – обеспечивает инициативу и возможность двойных ходов;
- Мудрость – раскрывает намерения врага.

Игрок может как выбрать специализацию, сосредоточив развитие на одной из характеристик, так и развивать сразу несколько. Например, можно создать героя, способного наносить мощные атаки за счёт высокой Силы, делать быстрые и частые ходы благодаря Ловкости, или строить стратегию на предвидении действий врага с опорой на Мудрость. А можно сделать героя, способного наносить не столь мощные атаки, зато делать их дважды при помощи Ловкости. А

3.2. АЛГОРИТМЫ ГЕНЕРАЦИИ КОМНАТ И КОРИДОРОВ

В рамках разработки подземелья для карточного рогалика была выбрана генерация на основе случайного блуждания с ограничениями. Такой подход обеспечивает псевдослучайное, но управляемое построение связной карты комнат и коридоров, подходящей для исследования.

Алгоритм генерации реализован в классе `DungeonGenerator`. Он создаёт комнаты последовательно, начиная с центральной с координатами (0,0), и добавляет новые в случайно выбранных соседних направлениях (север, юг, запад, восток), используя метод `GetNextCoords()`. При этом ведётся учёт уже занятых координат, чтобы избежать наложения комнат. Количество создаваемых комнат выбирается случайно в пределах, заданных в настройках `DungeonSettings`.

После генерации набора комнат формируется их связность: каждая комната проверяется на наличие соседей на расстоянии 1 клетки по сетке и соединяется с ними, что гарантирует, что каждая комната будет достижима. Коридоры создаются для каждой уникальной пары соединённых комнат. Такая система предотвращает изолированные зоны на карте и исключает тупики без события или награды.

Алгоритм генерации подземелья также включает назначение событий для каждой комнаты. Стартовая комната получает уникальное событие, зависящее от класса персонажа. В подземелье всегда есть 1 комната босса. Для остальных комнат используется система `EventData`, которая уже позволяет генерировать случайные события из различных категорий (боевые, хорошие, ловушки, классовые), с учётом заданных весов. Это создаёт разнообразие и позволяет гибко расширять пул ивентов для дальнейшего развития.

В целях обеспечения стабильной производительности и снижения нагрузки на систему при генерации и отображении подземелья в реальном времени была реализована динамическая подгрузка и выгрузка комнат и коридоров на основе позиции игрока. Когда игрок переходит в новую комнату, система создаёт комнаты, расположенные недалеко (в радиусе 2-х комнат) от текущего положения игрока и удаляет комнаты, которые расположены дальше. Пример работы продемонстрирован на рисунке 10. Красной стрелкой отмечено местоположение игрока, ракурс камеры не изменяется, меняется только местоположение игрока. Такая система особенно важна в 3D-играх, где избыточное количество активных объектов сцены может привести к просадкам FPS и увеличению времени отклика.

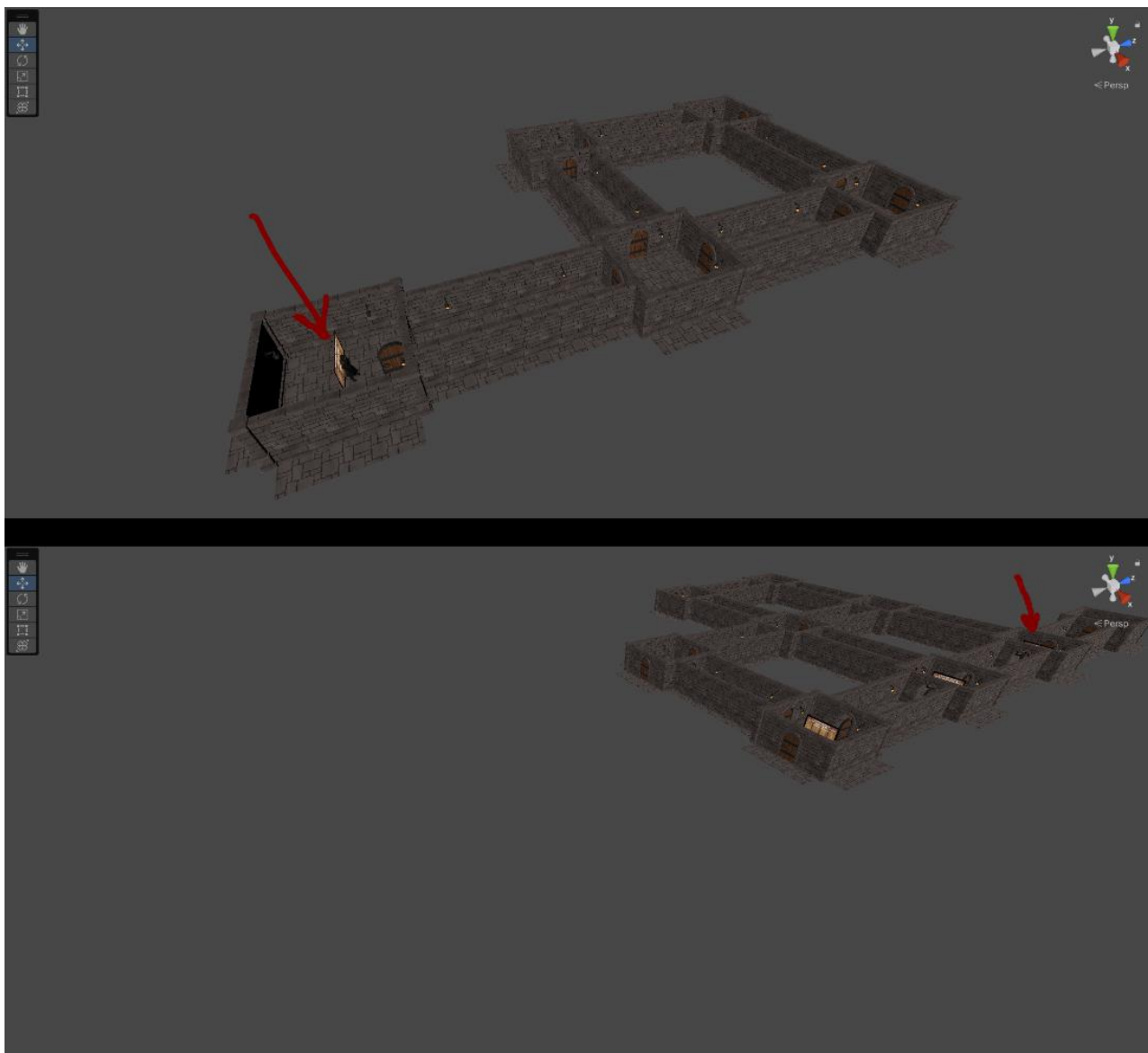


Рисунок 10 – Динамическая подгрузка комнат

Ключевая логика реализована в методе `BuildDungeonMap()`, который выполняется при запуске сцены и при необходимости обновления (перемещении игрока в другую комнату). Алгоритм работает следующим образом:

1. Загрузка комнат и коридоров в зоне видимости: комнаты и коридоры создаются только в пределах радиуса двух клеток от текущей комнаты (`MiniMapUI.currentRoom`). Это позволяет в любой момент видеть до 25 комнат и соответствующие им соединения, что достаточно для комфортного геймплея, не перегружая сцену лишними объектами.

2. Выгрузка удалённых объектов: все комнаты и коридоры, координаты которых выходят за пределы области интереса, уничтожаются с помощью `Destroy()`, а также удаляются из списков созданных комнат и коридоров. Перед этим их состояние (например, открыты ли двери, текущая стадия инвента) сохраняется в словари `deletedRooms` и `deletedCorridors`. Это позволяет при необходимости восстановить эти элементы в прежнем виде при повторной генерации.

3. Восстановление состояния объектов: при повторной генерации ранее удалённой комнаты или коридора проверяется, есть ли она в словаре удалённых элементов. Если да – её прежнее состояние восстанавливается.

4. Минимизация операций в кадре: все ключевые операции – как генерация, так и удаление объектов – производятся только при необходимости, когда флаг `isNeedUpdate` становится `true`. Это предотвращает лишние вычисления в каждом кадре и позволяет системе обновляться только в нужные моменты, например, при перемещении игрока.

Такой подход позволяет:

- Эффективно масштабировать карту без риска перегрузки;
- Сохранять визуальное и логическое состояние комнат;
- Гибко управлять уровнем детализации в зависимости от удалённости от игрока.

В текущей реализации подземелья предусмотрены два основных типа комнат:

- Обычная комната – базовый тип, в которой происходит одно из событий. Не обладает уникальными визуальными или геймплейными особенностями и составляет основную массу подземелья.
- Комната босса – выделяется визуально: окутана туманом и закрывается при входе. В ней игрок сталкивается с финальным противником. Содержит уникальное событие, связанное с боем против босса.

На рисунке 11 представлены текущие типы комнат. В дальнейшем планируется добавить дополнительные типы комнат, например комнату торговца, где игрок сможет улучшать характеристики и приобретать новые навыки, что расширит стратегические возможности и разнообразит прохождение.

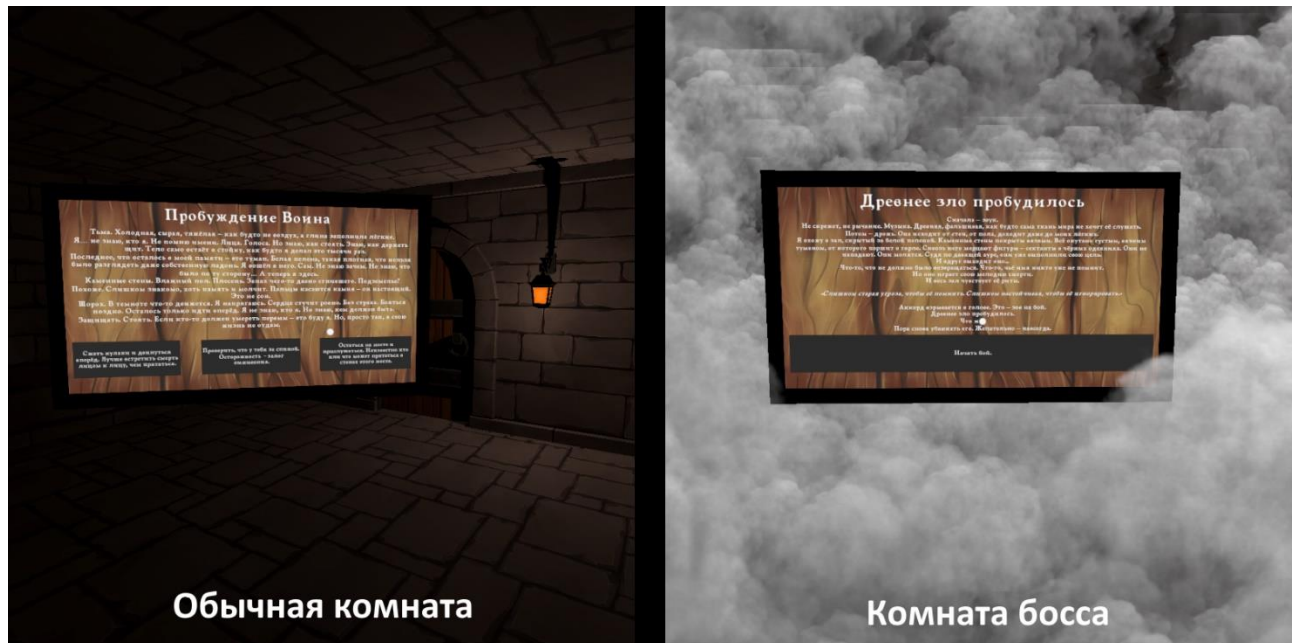


Рисунок 11 – Типы комнат

3.3. ИНТЕРАКТИВНЫЕ ЭЛЕМЕНТЫ ПОДЗЕМЕЛЬЯ

В подземелье реализована система интерактивных элементов, таких как двери, которые могут быть открыты или закрыты в зависимости от наличия ключа или выполнения условий. Взаимодействие с этими объектами осуществляется через систему лучей (Raycasting), а также с использованием Canvas для отображения пользовательского интерфейса.

Для реализации взаимодействия с объектами в мире используется система лучей, которая позволяет игроку взаимодействовать с объектами, находящимися в пределах его видимости. Главные компоненты:

- `Raycast.cs` – скрипт, который отслеживает объекты, с которыми можно взаимодействовать (например, двери, ключи). Луч, посылаемый от камеры, проверяет, есть ли на пути объект с тегом `"InteractiveObject"`.
- Система UI – для отображения кнопок взаимодействия используется `WorldSpaceCanvas`, который регистрирует свои элементы для обработки событий с помощью `Raycast`.

При наведении луча на интерактивный объект визуальный индикатор (`crosshair`) окрашивается в красный цвет, который сигнализирует о возможности взаимодействия. На рисунке 12 представлена разница между неинтерактивным и интерактивным объектом. Когда игрок нажимает левую кнопку мыши, выполняется соответствующее действие, например открытие или закрытие двери.



Рисунок 12 – Визуальный индикатор интерактивности

Двери в подземелье могут быть закрытыми, открытыми или заблокированными, в зависимости от того, есть ли у игрока соответствующий ключ. Для управления дверями используется система анимаций, которая запускает анимации открытия и закрытия дверей:

- `KeyDoorController.cs` – отвечает за управление состоянием дверей. Каждая дверь может быть открыта или закрыта, в зависимости от наличия ключа у игрока.
- `KeyItemController.cs` – управляет объектами, связанными с ключами (например, двери с ключом или сами ключи).

Двери, которые не могут быть открыты на данный момент, отображают модельку замка. Пример такой блокировки на рисунке 13. Если у игрока есть правильный ключ или он завершил событие в комнате, дверь открывается.

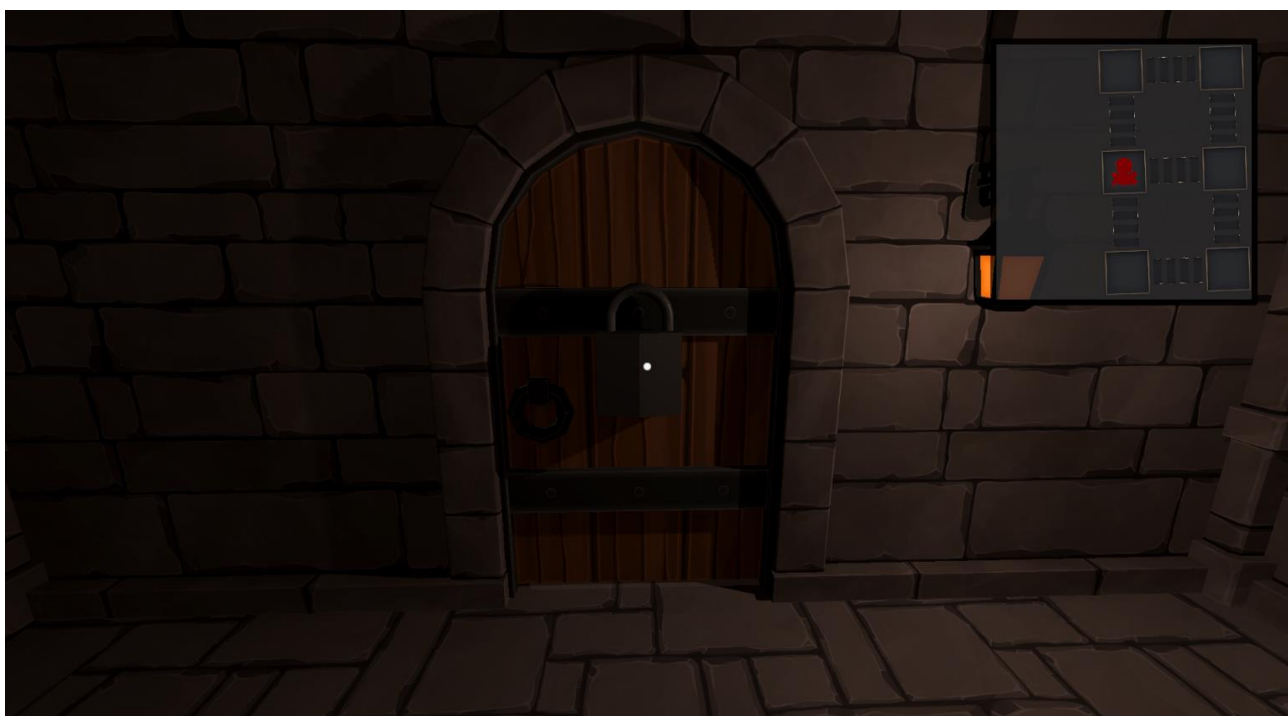


Рисунок 13 – Закрытая дверь

Если дверь требует определённого ключа для открытия, то замок на двери поменяет свой цвет на цвет ключа, который требуется для её открытия.

`KeyInventory.cs` – класс, который управляет состоянием ключей у игрока. Например, если у игрока есть `RedKey`, он может открыть двери с красным замком.

3.4. ТИПЫ СОБЫТИЙ И ИХ ОСОБЕННОСТИ

Событийная система игры построена на последовательности сменяющих друг друга этапов – событий, каждое из которых может содержать выбор, бой, награды или последствия для отряда. События представлены в виде узлов, связанных между собой, и управляют как повествованием, так и прогрессом игрока в пределах одной локации. Все выборы в событиях ведут к следующим этапам, включая отдельные финалы после боя или развилки.

Основные типы событий:

- Часть события – представляет собой базовую единицу события. Используется для построения линейных или разветвлённых сценариев. Особенностей не имеет, но служит связующим звеном между действиями и результатами. Часто служит заключительным событием;
- Начальное событие – сюжетная точка начала приключения для конкретного персонажа. Это обязательное событие, которое всегда появляется в стартовой комнате и служит вводом в игровой процесс. Его невозможно пропустить или заменить;
- Событие входа – активируется сразу после входа игрока в комнату. До фактического входа оно остаётся недоступным. Часто такое событие блокирует возможность немедленного выхода, заставляя игрока взаимодействовать с содержимым комнаты;
- Боевое событие – приводит к сражению с врагами. До окончания боя двери комнаты блокируются, не позволяя покинуть её преждевременно. Является важной частью тактического вызова и теста на готовность отряда;
- Событие босса – представляет ключевую схватку всей локации. В этом событии персонаж полностью изолирован: все выходы из комнаты удаляются до завершения боя;
- Победа над боссом – событие, активирующееся после победы над боссом. Является завершающей точкой локации. Выход из комнаты в этот момент

трактуются как успешное завершение текущего забега, что можно расценивать как хорошую концовку;

- Ловушка – особый тип события, требующий реализации двух разных исходов: попадания в ловушку или её избегания. Включать проверку характеристик. Добавляет напряжённость и вариативность исследованию.

Некоторые события содержат скрытые ветки развития, которые не отображаются в интерфейсе по умолчанию. Открытие таких скрытых вариантов зависит от характеристики мудрости персонажа:

- За каждую единицу мудрости персонаж получает +5% шанс на обнаружение скрытого выбора, вплоть до предела в 80%;
- Если шанс срабатывает, скрытый вариант отображается как дополнительная кнопка;
- Скрытые выборы часто предоставляют уникальные пути, бонусы или позволяют избежать нежелательных последствий.

Таким образом, развитие мудрости открывает альтернативные пути прохождения и делает персонажа более восприимчивым к деталям мира.

События типа ловушек включают проверку одной из характеристик, выбранной заранее в `EventData`. Проверка определяет исход события:

- При попадании в ловушку запускается проверка значения одной из характеристик: сила, ловкость, мудрость, телосложение или защита;
- За каждую единицу характеристики персонаж получает +5% шанс на избегание ловушки, вплоть до предела в 80%. Если шанс срабатывает, то считается, что персонаж избежал срабатывания ловушки;
- В случае успеха герой может продолжить путь без последствий или получить бонус;

- При провале ловушка срабатывает – возможны бои, потеря здоровья или другие негативные эффекты.

По завершении событий, персонаж может получить различные награды. Тип и объём награды задаются в `EventData` и могут включать:

- **Ресурсы:** души – основная валюта, используемая для торговли;
- **Здоровье:** восстановление части здоровья персонажа;
- **Характеристики:** повышение параметров – силы, ловкости, мудрости, телосложения и защиты;
- **Навыки:** получение нового умения. Может быть заранее определено (фиксированное умение) или выбрано случайно из указанного списка;
- **Тёмная карма:** особый параметр, отражающий тёмные решения героя.

Поддерживаются как фиксированные награды, так и случайные, с заданными диапазонами значений или шансами выпадения.

Отдельным важным аспектом является система тёмной кармы, представляющая собой параметр, отражающий морально неоднозначные или откровенно отрицательные выборы персонажа. Карма:

- Может быть увеличена при прохождении определённых событий, когда игрок выбирает жестокие, эгоистичные или сомнительные с моральной точки зрения варианты;
- Не имеет положительного аналога – наращивается только тёмная карма. Это подчеркивает, что нейтральный или светлый путь – это путь отказа от искушений, а не накопление "добра";
- Некоторые навыки становятся сильнее при высоком уровне кармы.

4. СИСТЕМА ХРАНЕНИЯ ДАННЫХ

4.1. ХРАНЕНИЕ ОСНОВНОЙ ИНФОРМАЦИИ О ЗАБЕГЕ

Во время прохождения подземелья информация о текущем состоянии игры сохраняется в структуре `RunInfo`. Это сериализуемый класс, содержащий все необходимые данные для восстановления и продолжения текущей игры. Данный подход обеспечивает устойчивость к перезапускам, а также удобство анализа и отладки.

Основные компоненты `RunInfo`:

- `saveTeam` – список объектов `CharacterSaveData`, представляющих состояние каждого персонажа: здоровье, характеристики, навыки, фаза, статус (жив/мертв);
- `PlayerTeam` – временный список объектов `PlayableCharacter`, используемый в рамках текущей сессии (не сериализуется);
- `badKarma` – текущий уровень тёмной кармы.
- `globalBuffs` – список активных глобальных эффектов, влияющих на весь отряд или окружение;
- `currentLocation` – текущая локация (в данный момент реализовано только Подземелье (`Dungeon`));
- `dungeonStructure` – структура подземелья, включающая комнаты, коридоры и связи между ними;
- `currentRoom` / `currentCorridor` – указатели на текущую комнату и коридор, где находится игрок;
- `positionX`, `positionY`, `positionZ` – мировые координаты персонажа;

- `rotationX, rotationY` – направление взгляда (ориентация камеры/модели);
- `activePotion, passivePotion` – были ли выпиты зелья от торговца, увеличивающие количество доступных слотов навыков;
- `souls` – текущее количество душ, накопленных в забеге;
- `slotID` – ID сохранённого слота, по которому можно восстановить прогресс.

Для сохранения индивидуальных данных каждого героя используется класс `CharacterSaveData`. Он хранит:

- Базовые параметры: здоровье, защита, сила, ловкость, мудрость, телосложение;
- Текущую фазу развития (`currentPhase`): 1 – человек, 2 – получеловек, 3 – монстр. Является заготовкой под механику тёмной кармы;
- Флаги `isDead` и `isSpawn` – для учёта состояний;
- Список навыков (`skillNameList`) – хранится по именам, что позволяет позже загружать соответствующие `SkillData`.

4.2. ХРАНЕНИЕ ИНФОРМАЦИИ О ПЕРСОНАЖАХ

В рамках разработки игры была реализована модульная система хранения и обработки информации о персонажах, основанная на использовании архитектурных решений, предоставляемых движком Unity. Основная цель этой подсистемы – обеспечить гибкость в описании игровых сущностей, расширяемость механик и удобство при интеграции новых персонажей в игровой процесс.

Базовым элементом хранения статических данных о персонажах служит `ScriptableObject` – контейнер данных, позволяющий создавать уникальные

экземпляры информации без необходимости дублирования кода или логики. Для каждого персонажа создаётся отдельный `ScriptableObject`-объект, содержащий ключевые поля: имя, описание, класс (например, воин, лучник и т.д.), визуальные ресурсы, стартовые характеристики, приоритеты в прокачке характеристик (для различных эффектов), а также списки активных и доступных классу навыков.

Для моделирования и хранения информации о персонажах, участвующих в боевой системе, используется объектно-ориентированный подход. Архитектура хранения данных построена на основе базового класса персонажа, а также специализированного производного класса, что обеспечивает расширяемость, повторное использование кода и удобную организацию логики поведения.

Базовая модель персонажа, представленная в виде класса `Fighter`, инкапсулирует все ключевые параметры и состояния, характерные для любого бойца – как союзного, так и враждебного. В эту структуру входят основные боевые характеристики (здоровье, защита, сила, ловкость, мудрость, выносливость), набор умений, список активных эффектов, система перезарядки умений и логика взаимодействия с боевым процессом. Кроме того, в классе реализованы механизмы инициализации данных на основе внешних источников (`ScriptableObject`) и функции активации пассивных эффектов при появлении на поле боя.

Для игровых персонажей был выделен специализированный подкласс `PlayableCharacter`, расширяющий базовый функционал. Он добавляет уникальные особенности, такие как стадия трансформации (человек, получеловек, монстр) – заготовка для системы тёмной кармы, проверки на доступные навыки и их расширение при некоторых глобальных навыках, а также использование различных портретов в зависимости от текущей стадии. Такое расширение даёт гибкость в реализации прогрессии персонажа и визуальной составляющей его развития.

Также реализована логика ограничения на количество умений, зависящая от характеристик персонажа и глобальных условий. Это создаёт дополнительную глубину в стратегическом управлении развитием героя и подбором умений.

Таким образом, система хранения информации о персонажах является модульной, гибкой и масштабируемой. Она сочетает в себе использование шаблонов данных, инкапсуляцию логики поведения, поддержку сохранения состояния и адаптивное изменение характеристик в бою.

4.3. ХРАНЕНИЕ ИНФОРМАЦИИ О НАВЫКАХ

Система навыков в проекте построена на чётком разделении данных и логики. Основу архитектуры составляют три ключевых компонента: `SkillSO`, `Skill` и `SkillDB`. Данные об умении, такие как его визуальное оформление, описание и характеристики, хранятся в `SkillSO` – это `ScriptableObject`, предназначенный для заполнения через Unity-инспектор. Именно здесь определяется название умения, его иконка, описание, а также сопроводительная цитата, если она нужна. Помимо эстетической информации, здесь же задаются и игровые параметры – тип действия (например, атака, лечение или наложение эффекта), тип цели (одиночная, массовая, пассивная и пр.), принадлежность к определённой стихии (физика, огонь, вода, тьма и т.п.), время восстановления, возможность применять умение к умершим персонажам, универсальность (подходит ли умение для всех персонажей), а также наличие флага, указывающего, является ли умение проклятием. Для пассивных умений предусмотрены поля, указывающие эффекты (хранятся в виде `enum`), активирующиеся как в начале боя, так и действующие постоянно.

Вся логика применения умения сосредоточена в классе `Skill`. Этот класс инкапсулирует поведение умения и хранит ссылку на соответствующий `SkillSO`. Поведение описывается делегатами – это позволяет гибко определять, как именно работает умение в бою. Делегат `cast` отвечает за основное применение умения к списку целей, а `calc` используется для предварительного вычисления числовых эффектов – урона, исцеления и т.д. Помимо этого, существуют делегаты, связанные с жизненным циклом персонажа: `death` (событие при смерти), а также `passive` и `reverse` – для постоянных эффектов, действующих,

пока персонаж жив, и их отмены после смерти. Некоторые из них не используются, поскольку система эффектов позволила отказаться от них. Методы `Description()` и `Cast()` в классе `Skill` учитывают активные модификаторы боя и глобальные модификаторы (хранящиеся в `RunInfo.GlobalBuff`).

Фактическое создание умения осуществляется через указание пути к нужному файлу данных: `"SkillData/Active/"` или `"SkillData/Passive/"`. В зависимости от того, является ли умение активным или пассивным, используется соответствующий путь.

Все экземпляры `Skill`, используемые в игре, централизованно управляются через `SkillDB` – синглтон, который служит базой данных всех умений. При инициализации игры он загружает все умения и обеспечивает доступ к ним по имени. Навыки добавляются вручную в базу с помощью специальных методов. Активные умения регистрируются через `AddSkillCast()`, пассивные с логикой – через `AddSkillPassive()`. Для событий при смерти используется метод `AddSkillDeath()`. При этом каждое умение создаётся с помощью конструктора `Skill(string name, bool isPassive)`, после чего настраивается необходимыми делегатами. При необходимости можно получить доступ к умению по имени через `GetSkillByName(name)`, либо загрузить список определённых скиллов из папки `Resources` при помощи `GetPoolByName(name)`.

Таким образом, архитектура навыков остаётся гибкой, масштабируемой и поддерживает разделение ответственности между данными и их поведением, что облегчает как разработку, так и настройку умений без необходимости изменения кода.

4.4. ХРАНЕНИЕ ИНФОРМАЦИИ О СОБЫТИЯХ

События в игре хранятся в виде `ScriptableObject` типа `EventData`. Каждое событие представляет собой одну из возможных ситуаций, встречающихся при прохождении подземелья: ловушки, бои, сюжетные повороты, выборы с последствиями и т.д. Структура хранения обеспечивает модульность, поддержку

линейных и разветвлённых сценариев, а также интеграцию с системой наград, боёв и ловушек.

EventData содержит следующую информацию:

- **eventID / eventID_Part** – уникальный идентификатор события и его части (например, 3.1, 3.2 и т.д.) в рамках своего типа;
- **eventName / eventText** – название и описание события. Отдельно может задаваться альтернативный текст для персонажа-женщины;
- **eventType** – тип события;
- **choices** – список следующих частей события (ветвление истории, выбор игрока);
- **textChoice / isHidden** – текст на кнопке выбора и признак скрытого выбора;
- **isLockableEvent** – блокирует ли событие выход из комнаты до завершения.

EventData поддерживает:

- Ловушки с проверкой характеристик (**checkTrap**), с результатами: срабатывание и избегание;
- Бои со списком врагов (**enemies, enemiesCount**) или задание параметров для их случайной генерации (**randomEnemies, enemiesChances** и др.);
- Флаги: **isFog** (враги неизвестны до боя), **isNotOver** (бой не завершает забег при поражении).

Награды:

- Души, характеристики, здоровье, тёмная карма;

- Навыки (из фиксированного списка или случайный из нужного хранилища);
- Поддержка случайных и фиксированных наград.

При загрузке комнаты, событие из `room.eventData` передаётся в метод `FillEvent()`, где:

- Проверяется тип события и условия его отображения (например, ловушки или стартовые события не отображаются, если это не текущая комната);
- Изменяется UI: название, текст, кнопки выбора;
- Применяются эффекты наград, если событие завершено (`choices.Count == 0`) и награда ещё не была получена;
- Расчёт награды производится на основе диапазонов, указанных в `EventData`, с применением случайности;
- Отображается текстовая строка с полученными ресурсами/характеристиками.

События могут быть связаны между собой через список `choices`, образуя древовидную структуру. Это позволяет реализовать:

- Простые линейные цепочки событий;
- Ветвящиеся сюжетные линии в зависимости от выбора игрока;
- Альтернативные исходы (`isNotOver` и поражение в бою → 2-й выбор, прошёл проверку на ловушке → 2-й выбор).

Примеры использования:

- Боевой ивент: `eventType = FightEvent`, список врагов, выбор один – "Начать бой";
- Ловушка: `eventType = Trap`, два варианта событий – попасть в ловушку или избежать, с проверкой по характеристике;

- Награда: если `choices.Count == 0`, применяются награды (души, навыки и т.д.), указанные в `EventData`.

4.5. ФОРМАТ СЕРИАЛИЗАЦИИ ДАННЫХ

В главном меню игры предусмотрены три слота для сохранений, что позволяет играть 3-м разным людям на одном компьютере или одному игроку если он захочет поиграть другим персонажем, не прерывая текущую игру. Каждый слот имеет свой уникальный файл сохранения с расширением `.corrupted`, что не только соответствует тематике игры, но и добавляет элемент загадочности, а также затрудняет попытки несанкционированного редактирования данных сохранений.

Если файл сохранения окажется повреждённым или некорректным, игра уведомит об этом игрока сообщением "повреждённые данные" в слоте сохранения. Это помогает избежать неправильной работы игры, если файл был изменён или повреждён по каким-либо причинам.

В системе сохранений игры используется ассет (библиотека) для `Unity – Json & MessagePack Serialization`. Он выполняет сериализацию данных, основанную на двух технологиях: `Json` и `MessagePack`. Эти подходы позволяют эффективно сохранять и загружать данные, обеспечивая быстрый доступ и низкие затраты памяти. Для достижения большей производительности и безопасности, из этих двух подходов для проекта был выбран `MessagePack`.

`MessagePack` – это бинарный формат сериализации, который эффективно упаковывает объекты. Он более производителен в сравнении с традиционными текстовыми форматами сериализации, такими как `JSON` или `XML`. `MessagePack` позволяет работать с бинарными данными напрямую, что ускоряет процессы записи и чтения данных.

Для защиты данных сохраняется контрольная сумма `HMAC (Hash-based Message Authentication Code)`. Она записывается в первые 32 байта файла сохранения, что позволяет впоследствии проверять целостность данных. После расчета, контрольная сумма записывается в файл, а затем идут зашифрованные

данные. При загрузке прогресса эти данные проверяются на корректность: вычисляется контрольная сумма для зашифрованной части, после чего она сравнивается с сохранённой в файле.

Для сохранения прогресса создаются все необходимые данные игры, такие как состав команды игрока и текущее положение персонажа, после чего данные сериализуются с использованием `MessagePack`. Сначала сериализуются данные с помощью `MsgPack.Serialize()`, затем эти данные шифруются с использованием алгоритма AES. После шифрования вычисляется контрольная сумма HMAC для зашифрованных данных, которая записывается в файл перед самими данными.

Когда игрок загружает сохранение, происходит следующее: сначала считывается файл, из которого извлекаются HMAC и зашифрованные данные. Сначала проверяется контрольная сумма, и, если она не совпадает с рассчитанной, данные считаются поврежденными. В случае успешной проверки данных, происходит их дешифровка, и затем данные десериализуются в объект с помощью `MsgPack.Deserialize()`. После этого данные восстанавливаются в том виде, в каком они были в момент сохранения, включая команду игроков и их характеристики.

4.6. ЗАЩИТА ФАЙЛОВ СОХРАНЕНИЙ ОТ ИЗМЕНЕНИЙ

Для предотвращения подмены или ручного редактирования файлов сохранений, в проекте реализована надёжная система защиты, основанная на двух криптографических механизмах: шифрование по алгоритму AES и контроль целостности с помощью HMAC. Эта система обеспечивает конфиденциальность и гарантирует, что содержимое файла не было изменено после создания.

AES (Advanced Encryption Standard) – это симметричный блочный алгоритм шифрования, стандартизированный NIST. Он широко используется благодаря своей скорости и высокой степени безопасности. В данном проекте

используется реализация `Aes.Create()` из `.NET`, которая по умолчанию применяет AES-256 с режимом CBC и паддингом PKCS7.

AES-256 означает, что для шифрования используется ключ длиной 256 бит (32 байта). Это один из самых надёжных и стойких вариантов AES, устойчивый к атаке полного перебора (*brute-force*), так как общее количество возможных ключей составляет 2^{256} , что делает подбор невозможным.

Режим CBC (*Cipher Block Chaining*) – один из классических режимов работы блочных шифров. Он шифрует каждый блок данных не отдельно, а с учётом результата предыдущего блока. Это достигается за счёт того, что перед шифрованием каждый блок выполняет операцию XOR с предыдущим зашифрованным блоком. Для первого блока используется специальный вектор инициализации (IV), который должен быть уникальным и случайным для каждого зашифрованного файла. Благодаря этому один и тот же текст при повторном шифровании с тем же ключом, но разным IV, даст совершенно разный результат.

Паддинг PKCS7 используется для выравнивания данных перед шифрованием, так как AES работает с фиксированной длиной блока – 16 байт. Если последние байты данных не заполняют весь блок, то в конец добавляются специальные байты, каждый из которых содержит число, равное количеству добавленных байт. Например, если не хватает 5 байт, в конец добавляется: 05 05 05 05 05. Это позволяет при расшифровке однозначно определить и удалить паддинг, восстанавливая исходные данные.

В `.NET` это всё настраивается автоматически при использовании `Aes.Create()`, и паддинг PKCS7 применяется по умолчанию. Таким образом, не требуется вручную управлять выравниванием данных – алгоритм сам добавит и удалит паддинг при шифровании и расшифровке соответственно.

Для каждого сохранения применяется пара ключ/вектор инициализации (IV), которые хранятся и извлекаются через `KeyManager.GetSavedKeys()`.

Данные сериализуются в бинарный формат с помощью `MessagePack`, после чего записываются в поток `MemoryStream`.

Поток передаётся в `CryptoStream`, обернутый поверх `MemoryStream`, через `aes.CreateEncryptor()`, что обеспечивает шифрование при записи.

Результат шифрования возвращается в виде массива байтов и готов к сохранению на диск.

Пример реализации:

```
using Aes aes = Aes.Create();
aes.Key = key;
aes.IV = iv;
using CryptoStream cs = new(ms, aes.CreateEncryptor(),
CryptoStreamMode.Write);
```

Аналогично, при загрузке сохранения происходит расшифровка через `aes.CreateDecryptor()` с тем же ключом и IV. Если ключи не совпадают или файл повреждён, расшифровка завершится с ошибкой, предотвращая использование некорректных данных.

HMAC-SHA256 (Hash-based Message Authentication Code) – это механизм для проверки целостности и подлинности данных с использованием хеш-функции SHA-256 и секретного ключа. HMAC устойчив к атакам, при которых злоумышленник может подделать сообщение с тем же хешем. Это достигается благодаря включению в вычисление хеша секретного ключа, известного только доверенным сторонам.

Алгоритм HMAC работает следующим образом:

1. Если ключ длиннее блока хеш-функции (64 байта для SHA-256), он хешируется;
2. Ключ дополняется нулями до длины 64 байта;
3. Вычисляются два значения – внутренний и внешний паддинги:

$$\text{ipad} = \text{key} \text{ XOR } 0x36, \quad (1)$$

$$\text{opad} = \text{key} \text{ XOR } 0x5C; \quad (2)$$

4. Рассчитывается внутренний и внешний хеш:

$$\text{innerHash} = \text{SHA256}(\text{ipad} \parallel \text{data}), \quad (3)$$

$$\text{HMAC} = \text{SHA256}(\text{opad} \parallel \text{innerHash}). \quad (4)$$

В проекте используется встроенный класс `HMACSHA256` из `.NET`, который реализует всё описанное выше. В методе `ComputeHMAC(byte[] data, byte[] key)`:

- `new HMACSHA256(key)` – создаёт экземпляр HMAC с указанным секретным ключом `hmacKey`;
- `ComputeHash(data)` – возвращает итоговую HMAC-сумму длиной 32 байта, вычисленную по зашифрованным данным.

Даже если злоумышленник попытается отредактировать бинарный файл сохранения, он не сможет создать корректный HMAC без знания `hmacKey`, и слот будет считаться повреждённым.

Шифрование AES предотвращает возможность анализа структуры сохранения, что дополнительно защищает от реверс-инжиниринга.

Использование криптографии на уровне сериализации и записи делает систему надёжной и пригодной для использования даже в онлайн-сценариях (при необходимости).

Таким образом, защита файлов сохранений строится на двух уровнях:

- AES – скрывает содержимое файла и защищает от анализа;
- HMAC – гарантирует неизменность и целостность данных.

Для работы шифрования и HMAC необходимы криптографические ключи: `aesKey`, `aesIV` и `hmacKey`. В проекте реализован специальный класс `KeyManager`, который отвечает за генерацию, хранение и извлечение этих ключей. Все ключи хранятся в `PlayerPrefs` – встроенном хранилище Unity для сохранения пользовательских данных между сессиями.

Ключи создаются при помощи криптографически безопасного генератора случайных чисел `RNGCryptoServiceProvider`, который гарантирует высокую

энтропию и непредсказуемость. При вызове метода `GenerateAndSaveKeys()` генерируются:

- `aesKey` – 256-битный ключ (32 байта) для шифрования AES-256;
- `aesIV` – 128-битный вектор инициализации (16 байт) для режима CBC;
- `hmacKey` – 256-битный ключ (32 байта) для HMAC-SHA256.

Для хранения криптографических ключей в этом проекте используется `PlayerPrefs`, однако сами ключи сохраняются не в открытом виде. Каждому типу ключа (AES-ключ, вектор инициализации IV и HMAC-ключ) присваивается уникальное имя, хэшированное с добавлением суффикса, зависящего от устройства. Сами значения ключей предварительно шифруются при помощи AES на основе производного ключа, вычисленного из уникального идентификатора устройства (`SystemInfo.deviceUniqueIdentifier`). Это обеспечивает дополнительную защиту от прямого доступа к данным даже при компрометации `PlayerPrefs`. Ключи генерируются случайно, шифруются с использованием AES-256 в режиме CBC с PKCS7-паддингом и сохраняются в `PlayerPrefs` в виде строки Base64. Они расшифровываются обратно перед использованием. Такой подход повышает безопасность хранения информации, сохраняя при этом простоту работы с системными средствами Unity.

5. ПОТЕНЦИАЛ ПОВТОРНОГО ИСПОЛЬЗОВАНИЯ

5.1. ГИБКОСТЬ БОЕВОЙ СИСТЕМЫ И РАСШИРЯЕМОСТЬ НАВЫКОВ

Боевая система в проекте спроектирована с учётом дальнейшей расширяемости и лёгкой модификации без необходимости менять базовую структуру игры. Навыки, эффекты и параметры бойцов реализованы через сериализуемые классы и `ScriptableObject`-объекты (`CharacterSO`, `SkillSO`), что позволяет легко добавлять новых персонажей и способностей.

Каждый навык описывается как отдельный объект данных, в котором указываются тип действия, целевая группа, условия активации и побочные эффекты. При выполнении навыка боевой движок просто вызывает соответствующую реализацию поведения, не полагаясь на жёстко закодированные условия.

Также предусмотрена поддержка глобальных и локальных эффектов: например, эффекты, действующие на всю команду или на отдельного бойца, и их взаимодействие друг с другом. Это реализуется через хранение усилений, наложенных на персонажа, что позволяет добавлять новые типы эффектов (яд, кровотечение, защита, временные модификаторы) без изменения ядра системы боя.

Система не привязана жёстко к порядку инициативы — ход может назначаться вручную или по заранее определённым правилам, что облегчает реализацию особых механик вроде "перехвата хода", "заставить врага пропустить ход" и других ситуационных навыков. Такая архитектура идеально подходит для игр в жанре тактических JRPG и карточных рогаиков, где важно обеспечить богатое взаимодействие между способностями и состояниями.

Система навыков построена таким образом, чтобы упрощать процесс добавления и настройки новых умений без необходимости сильно модифицировать существующий код. Каждый навык представлен как отдельный `SkillSO` —

ScriptableObject, содержащий метаданные (название, описание, иконку) и настройки эффектов, которые он вызывает при активации.

Добавление нового навыка включает следующие шаги:

1. Создание нового SkillSO: В Unity через контекстное меню создаётся новый SkillSO;
2. Задаются параметры: имя навыка, тип (атака, защита, лечение и т.д.), целевая категория (враг, союзник, все), визуальные параметры (иконка);
3. В классе SkillDB добавляется новая запись в метод InitializeSkillDatabase(). Если это пассивный навык, то часто кроме названия регистрировать ничего не нужно. Если активный, то требуется написать в классе SkillDB метод *skill name*Cast(). Если навык содержит переменные в описании (например: “Наносит {0} урона”), то требуется добавить метод *skill name*Calc(), рассчитывающий это значение;
4. Для пассивных навыков часто требуется также добавить эффект в класс Fighter.Buff или RunInfo.GlobalBuff и указать его в SkillSO.
5. Для пассивных навыков требуется обычно написать их обработку непосредственно в нужном месте кода. Например, в классе Fight в методе FightStart():

```
foreach (Fighter character in AllCharacter)
{
    foreach (var buff in character.buffs)
    {
        switch (buff)
        {
            case Fighter.Buff.QuickRebuff:
                character.CastSkill(new List<Fighter>() {
RandomEnemy(character) }, SkillDB.Instance.GetSkillByName("Basic
Attack"), false);
                break;
            case Fighter.Buff.EchoHope:
                int heal = (int)(character.max_hp * 0.15f);
                if (heal == 0) heal = 1;
                character.TakeHeal(heal);
                break;
            case Fighter.Buff.NoAttack:
                var skill = SkillDB.Instance.GetSkillByName("Basic Attack");
                if (character.skillsBattle.Contains(skill))
                {
                    character.skillsBattle.Remove(skill);
                }
            }
        }
    }
```

```

        }
        break;
    }
}

```

Такой подход обеспечивает максимальную модульность — основу навыков можно редактировать в редакторе Unity без правки логики. Если навык уже реализован в коде, то там его править редко приходится. Также такой подход позволяет реализовать любой нужный навык.

5.2. УНИВЕРСАЛЬНОСТЬ СИСТЕМЫ СОБЫТИЙ

Система событий в игре построена с расчётом на масштабируемость и гибкость повествования. Каждое событие представляет собой отдельный объект (EventData), хранящийся в ресурсах (Resources/EventData) и содержащий все необходимые данные: текст описания, тип, варианты выбора и последствия каждого из них.

Ключевые особенности:

- **Структурированное хранение:** события разделены по категориям в папках (Class Events, Combat Events, Good Events, Trap Events, Start Events, Boss Events), что позволяет легко управлять ими в рамках процедурной генерации;
- **Расширяемость:** каждое событие может содержать произвольное количество вариантов выбора, каждый из которых задаётся через ту же структуру EventData, где тип события указан как Event Part;
- **Поддержка логики и последствий:** система поддерживает гибкие последствия — изменение характеристик, добавление или удаление навыков, лечение, запуск боёв или даже другие события.

Пример события: Тусклый огонь.

Тип события: сюжетное / классовое (Воин).

Контекст: встречается в подземелье, только для класса "Воин". Может быть частью линейки теневых событий.

Описание (текст события): представлен на рисунке 14.

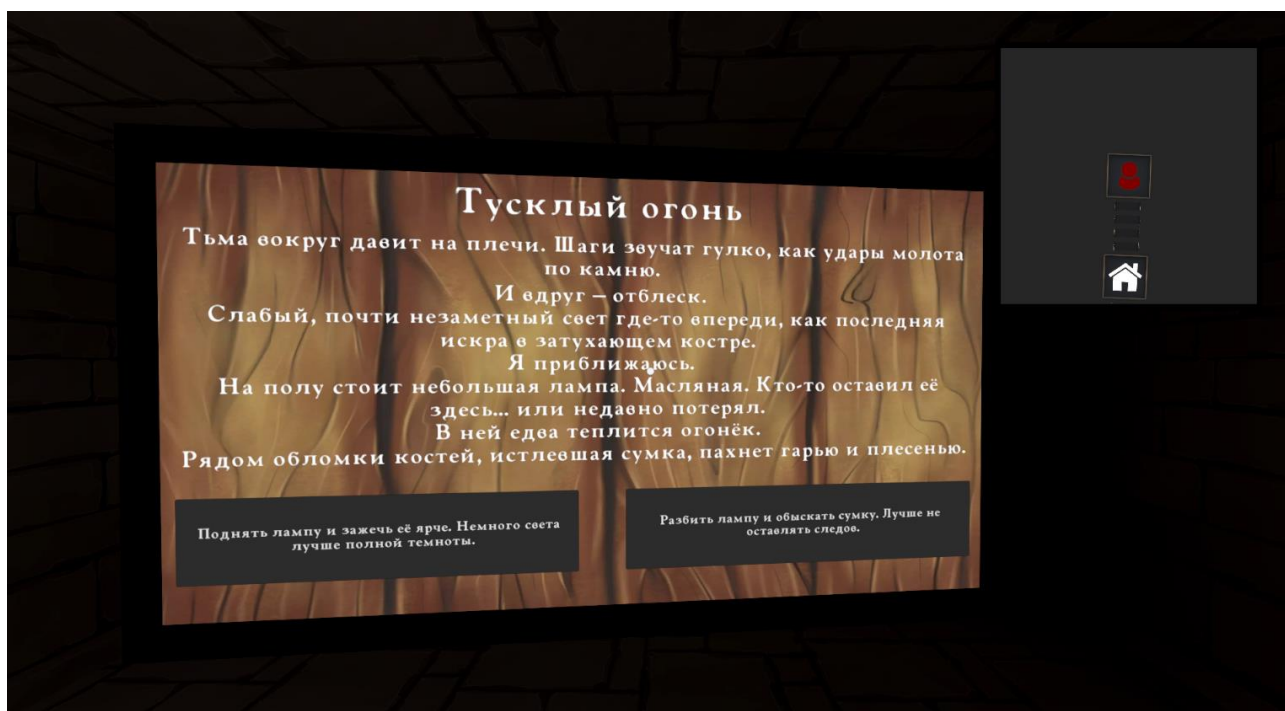


Рисунок 14 – Текст события

Варианты выбора:

- Поднять лампу и зажечь её ярче. Немного света лучше полной темноты.

Последствия:

- Начинается бой с мини-боссом ("Тень странника").
- После победы – развилка:
 - Принять тьму:
 - Получаете пассивный навык «Сердце тьмы»: ваши характеристики усиливаются от совершённых злых поступков.
 - Открывает доступ к навыкам стихии "Тьма".
 - Даёт случайный активный навык "Тьмы", если есть свободный слот.

- Тёмная карма значительно возрастает.
- Отказаться от тьмы:
 - Получаете пассивный навык «Проклятая рука»: обычная атака накладывает случайное проклятие.
 - Получаете активный навык "Лиловый туман", если есть свободный слот.
 - Тёмная карма немного возрастает.
- Разбить лампу и обыскать сумку. Лучше не оставлять следов. Последствия:
 - Получаете небольшое количество душ.
 - Тёмная карма немного увеличивается.
 - Событие завершается без боя.
- Обойти стороной. Здесь слишком много неправильного. Условие отображения:
 - Скрытое. Требуется проверка мудрости персонажа: шанс увеличивается с ростом характеристики.
- Последствия:
 - Получаете +1 к мудрости.
 - Событие завершается без боя.

Комментарий по реализации. Событие реализовано через универсальную систему `EventData`, загружаемую из `Resources`. Оно относится к категории `Class Events` и поддерживает скрытые варианты выбора, условия отображения, а также последствия в виде изменения характеристик, получения навыков, кармы или запуска боя. После боя возможна развилка через вложенные события (`Event Part`), реализующие разные линии развития.

5.3. ВОЗМОЖНОСТИ АДАПТАЦИИ ПРОЕКТА ПОД ДРУГИЕ ИГРОВЫЕ ЖАНРЫ

Несмотря на изначальную реализацию проекта в формате карточного рога-лика с исследованием подземелий, боевая система, архитектура событий и система навыков разработаны с высокой степенью абстракции и универсальности. Это позволяет адаптировать основные механики проекта под другие игровые жанры без кардинальной переработки кода. Например:

- Пошаговая тактическая игра с расстановкой юнитов (пример такой игры: *Darkest Dungeon*). Почему подходит: реализована механика отрядов персонажей, навыки, пошаговая система и события. Добавление сетки или линий фронта можно внедрить поверх существующей логики. Событийность и атмосфера только усилят глубину.
- Повествовательное приключение с выборами (пример таких игр: *Roadwarden*, *Reigns*, или даже визуальные новеллы с прогрессией). Почему подходит: система событий уже очень сильная и гибкая. Если убрать/упростить бои – получится глубокая текстовая игра с последствиями, прокачкой и ветвлениями.
- Стратегическая игра с элементами выживания (пример такой игры: *Curious Expedition*). Почему подходит: система процедурных комнат, событий и характеристик персонажей может лечь в основу выживания, сбора ресурсов, принятия решений. Бои можно упростить или оставить как есть.
- Игра с авто-боями и минимальным управлением (жанр игр – *Auto Chess*). Почему подходит: можно адаптировать текущую систему умений и характеристик под автоматические фазы боя, а построение отряда вынести в отдельную фазу.

Хотя архитектура проекта построена с прицелом на универсальность, есть случаи, когда адаптация может потребовать значительных изменений. Это касается ситуаций, где базовая логика – пошаговые бои, нарративные события и

система навыков – не вписывается в структуру новой игровой модели, даже несмотря на схожие жанры. В таких случаях необходимо перерабатывать фундаментальные модули, чтобы сохранить целостность игрового процесса и пользовательского опыта. Например:

- Рогалик с real-time боевой системой (пример таких игр: Hades, Dead Cells). Проблема: текущая система боя – строго пошаговая, с чёткой очередностью и пошаговыми эффектами. Чтобы перейти к экшену в реальном времени, нужно переделать абсолютно всю боевую логику, а также интерфейс умений и взаимодействие с врагами. Высокая сложность из-за конфликта геймплейных основ.
- Тактический рогалик с перемещением по сетке (Пример такой игры: Into the Breach). Проблема: отсутствие системы координат/тайлов, логика боя не учитывает позиционирование. Добавление перемещения, зон действия умений и окружения потребует создания нового слоя логики. Средняя сложность из-за требования архитектурной перестройки боевых сцен.
- Ролевая игра с открытым миром (игры жанра RPG). Проблема: текущая система событий и процедурная генерация построена на линейной прогрессии и изолированных комнатах. Адаптация к открытому миру потребует постоянной поддержки мира в памяти, систем синхронизации событий, перемещений, квестов. Высокая сложность из-за требования абсолютно новой структуры подземелья.
- ККИ с PvP/сетевыми элементами (пример такой игры: Hearthstone). Проблема: текущая система не рассчитана на синхронизацию состояния между игроками, нет строгих правил очередности и проверки состояния в онлайне. Механика "кармы" и случайных выборов плохо ложатся на конкурентную PvP-сцену, так что их пришлось бы адаптировать или вовсе убирать из игры. Высокая сложность из-за различия в базовом геймдизайне жанров и сложности сетевой реализации.

ЗАКЛЮЧЕНИЕ

В рамках данной работы был разработан прототип игры в жанре карточного рогалика, сочетающий 3D-исследование подземелий от первого лица и пошаговые 2D-бои с использованием карточной боевой системы. Проект имеет целостную структуру игрового цикла: от главного меню до системы сражений.

Одним из ключевых достижений можно считать гибкость и масштабируемость реализованных систем. Система событий позволяет легко добавлять новые сценарии с выбором и последствиями, а система боя с навыками-картами показывает себя очень универсальной. Она позволяет создавать уникальные умения, не меняя ядро логики. Это открывает возможности для повторного использования проекта в будущих разработках схожих жанров.

Особое внимание было уделено технической стороне реализации. В проекте использовалась система генерации частиц для создания атмосферы (эффектов) в комнате босса. Такой подход позволил создать выразительные сцены, не перегружая производительность. Для сохранения прогресса используется бинарная сериализация с помощью `MessagePack`, данные шифруются по алгоритму AES и сопровождаются HMAC-подписью для проверки целостности. Такой подход обеспечивает как компактность хранения, так и надёжную защиту сохранений от изменения.

Важно отметить, что за исключением визуальных 3D-моделей, использованных для графического оформления и ассета для использования `MessagePack`, все остальные системы были разработаны самостоятельно – включая генерацию, UI, механику боя, обработку событий и систему сохранений. Отказ от использования сторонних ассетов в логической части обеспечил глубокое понимание внутренних процессов и позволил создать модульную, легко расширяемую архитектуру.

СПИСОК ЛИТЕРАТУРЫ

1. Джозеф Хокинг. Unity в действии. Мультиплатформенная разработка на C#. – 3-е изд. – СПб.: Издательский дом «Питер», 2023. – 448 с.
2. Джесси Шелл. Геймдизайн. Как создать игру, в которую будут играть все. – 2-е изд. – Москва: Альпина Паблишер, 2021. – 640 с.
3. Слава Грис. Ошибки разработчиков видеоигр. От идеи до провала. – Москва: Издательство АСТ, 2023. – 320 с.
4. Unity Documentation. URL: <https://docs.unity3d.com/ru/530/Manual/index.html> (Дата обращения 14.05.2025).
5. Robert Nystrom. Game Programming Patterns. – Genever Benning, 2014. – 354 с.
6. Шнайер Брюс. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си. – Москва: Триумф, 2012. – 815 с.
7. Бонд Джереми Гибсон. Unity и C#. Геймдев от идеи до реализации. – 2-е изд. – СПб.: Издательский дом «Питер», 2022. – 928 с.
8. Дикинсон Крис. Оптимизация игр в Unity 5. Советы и методы оптимизации приложений. – Москва: ДМК-Пресс, 2017. – 306 с.
9. Let's Build a Roguelike. URL: <https://users.freebasic-portal.de/rdc/tutorials.html> (Дата обращения 14.05.2025).
10. Rayne Hall. Writing Dark Stories (Writer's Craft). – CreateSpace Independent Publishing Platform, 2014. – 140 с.

ПРИЛОЖЕНИЕ А. РЕПОЗИТОРИЙ

Программа расположена по ссылке:

https://disk.yandex.ru/d/VWMo2-u_MH3qoQ

Проект расположен здесь:

<https://github.com/JackRaycoon/Diplome>

В папке Assets/Scripts расположен код.

ПРИЛОЖЕНИЕ Б. ФАЙЛ DUNGEONGENERATOR.CS

Файл `DungeonGenerator.cs` – основной скрипт, отвечающий за процедурную генерацию и отображение подземелья в 3D-пространстве. Он создаёт комнаты и коридоры на основе настроек, размещая их в пределах видимости от текущей комнаты игрока. Скрипт динамически загружает и выгружает объекты, чтобы оптимизировать производительность и сохранить состояние (например, открытые двери). Также он иницирует мини-карту, восстанавливает элементы при повторной генерации, и обрабатывает события, связанные с комнатами. Начальная комната содержит уникальное событие, связанное с первым героем игрока.

```
using KeySystem;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using Unity.VisualScripting;
using UnityEngine;
using UnityEngine.UIElements;
using Random = UnityEngine.Random;

public class DungeonGenerator : MonoBehaviour
{
    [SerializeField] DungeonSettings settings;
    [SerializeField] GameObject roomPrefab;
    [SerializeField] GameObject bossRoomPrefab;
    [SerializeField] GameObject corridorPrefab;
    [SerializeField] float offsetRoom;
    [SerializeField] float offsetCorridor;
    [SerializeField] MiniMapUI miniMap;

    [SerializeField] GameObject player;

    public Loading loader;

    private List<Room> createdRooms = new();
    private List<GameObject> createdRoomsGO = new();
    private List<Corridor> createdCorridors = new();
    private List<GameObject> createdCorridorsGO = new();
    //private Dictionary<Vector2Int, RoomState> deletedRooms = new();
    private List<Room> deletedRooms = new();
    //private Dictionary<Corridor, CorridorState> deletedCorridors = new();
    private List<Corridor> deletedCorridors = new();

    public static bool isNeedUpdate = false;

    private Dungeon dungeon;

    private void Awake()
    {
        SaveLoadController.Load();
    }
    private void Start()
    {
        isNeedUpdate = false;
    }
}
```

```

var structure = SaveLoadController.runInfo.dungeonStructure;
if (SaveLoadController.runInfo.dungeonStructure == null)
{
    CreateDungeon(settings);
    structure = SaveLoadController.runInfo.dungeonStructure;
}
else
{
    structure.roomToConnectedRooms = ConnectRooms(structure.rooms);
}
dungeon = new Dungeon(structure);
BuildDungeonMap(true);
miniMap.CreateDungeonMiniMapUI(dungeon);
}

private void Update()
{
    if (isNeedUpdate)
    {
        isNeedUpdate = false;
        BuildDungeonMap(false);
    }
}

//Строит подземелье вокруг currentRoom и очищает всё остальное
private void BuildDungeonMap(bool firstBuild)
{
    var currentRoom = SaveLoadController.runInfo.currentRoom;
    if(currentRoom == null && firstBuild)
    {
        currentRoom = SaveLoadController.runInfo.currentCorridor.room1;
    }

    if (currentRoom == null) return;
    foreach (var room in dungeon.rooms)
    {
        if (createdRooms.Contains(room)) continue;
        if (Math.Abs(currentRoom.Coords.x - room.Coords.x) > 2
            || Math.Abs(currentRoom.Coords.y - room.Coords.y) > 2) continue;
        //Создаём комнаты, которые в пределах пары комнат от текущей
        RoomBehaviour roomBeh;
        roomBeh = room.roomType switch
        {
            Room.RoomType.Shop => throw new NotImplementedException(),
            Room.RoomType.Boss => Instantiate(bossRoomPrefab, new Vector3(offsetRoom *
room.Coords.x, -0.01f, offsetRoom * room.Coords.y),
Quaternion.identity,
transform).GetComponent<RoomBehaviour>(),
            _ => Instantiate(roomPrefab, new Vector3(offsetRoom * room.Coords.x, -0.01f,
offsetRoom * room.Coords.y),
Quaternion.identity,
transform).GetComponent<RoomBehaviour>()
        };
        /*roomBeh = Instantiate(roomPrefab, new Vector3(offsetRoom * room.Coords.x, -
0.01f, offsetRoom * room.Coords.y),
Quaternion.identity,
transform).GetComponent<RoomBehaviour>();*/
        roomBeh.gameObject.GetComponent<Room3D>().room = room;
        roomBeh.gameObject.GetComponent<Room3D>().loader = loader;
        createdRooms.Add(room);
        createdRoomsGO.Add(roomBeh.gameObject);
        bool[] status = new bool[4];
        foreach (var connectedRoom in dungeon.roomToConnectedRooms[room])
        {
            if (room.Coords.y - connectedRoom.Coords.y < 0) //North
            {
                status[0] = true;
            }
            if (room.Coords.y - connectedRoom.Coords.y > 0) //South
            {
                status[1] = true;
            }
        }
    }
}

```

```

    }
    if (room.Coords.x - connectedRoom.Coords.x < 0) //East
    {
        status[2] = true;
    }
    if (room.Coords.x - connectedRoom.Coords.x > 0) //West
    {
        status[3] = true;
    }
}
roomBeh.status = status;
roomBeh.UpdateRoom();

//Восстанавливаем состояние (пока только двери)
roomBeh.LoadDoorState(room.doorOpened);
/*if (deletedRooms.Contains(room))
{
    roomBeh.LoadDoorState(room.doorOpened);
}*/
}
//Удаляем комнаты, которые слишком далеко от игрока
for(int i = 0; i<createdRoomsGO.Count; i++)
{
    Room room = createdRoomsGO[i].GetComponent<Room3D>().room;
    if (Math.Abs(currentRoom.Coords.x - room.Coords.x) > 2
        || Math.Abs(currentRoom.Coords.y - room.Coords.y) > 2)
    {
        if (deletedRooms.Contains(room)) deletedRooms.Remove(room);
        deletedRooms.Add(room);
        createdRooms.Remove(createdRoomsGO[i].GetComponent<Room3D>().room);
        Destroy(createdRoomsGO[i]);
        createdRoomsGO.Remove(createdRoomsGO[i]);
        i--;
    }
}

//Создаём коридоры
foreach (var corridor in dungeon.corridors)
{
    if (createdCorridors.Contains(corridor)) continue;
    if (Math.Abs(currentRoom.Coords.x - corridor.room1.Coords.x) > 2
        || Math.Abs(currentRoom.Coords.y - corridor.room1.Coords.y) > 2
        || Math.Abs(currentRoom.Coords.x - corridor.room2.Coords.x) > 2
        || Math.Abs(currentRoom.Coords.y - corridor.room2.Coords.y) > 2) continue;
    float xMax = Mathf.Max(corridor.room2.Coords.x, corridor.room1.Coords.x);
    float xOffset = Mathf.Abs(corridor.room2.Coords.x - corridor.room1.Coords.x);
    float xFinalOffset = offsetRoom * xMax - offsetCorridor * xOffset;

    float yMax = Mathf.Max(corridor.room2.Coords.y, corridor.room1.Coords.y);
    float yOffset = Mathf.Abs(corridor.room2.Coords.y - corridor.room1.Coords.y);
    float yFinalOffset = offsetRoom * yMax - offsetCorridor * yOffset;

    GameObject go = Instantiate(corridorPrefab, new Vector3(
        xFinalOffset,
        0,
        yFinalOffset),
        Quaternion.Euler(0, corridor.orientation == CorridorOrientation.Horizontal ?
90 : 0, 0),
        transform);
    createdCorridors.Add(corridor);
    createdCorridorsGO.Add(go);
    go.GetComponent<Corridor3D>().corridor = corridor;
    //Восстанавливаем состояние
    if (deletedCorridors.Contains(corridor))
    {
        Debug.Log("Здесь как-то сделаем восстановление коридоров");
    }
}

//Удаляем коридоры, которые слишком далеко от игрока
for (int i = 0; i < createdCorridorsGO.Count; i++)

```



```

        {
            Corridor corridor = createdCorridorsGO[i].GetComponent<Corridor3D>().corridor;
            if (Math.Abs(currentRoom.Coords.x - corridor.room1.Coords.x) > 2
                || Math.Abs(currentRoom.Coords.y - corridor.room1.Coords.y) > 2
                || Math.Abs(currentRoom.Coords.x - corridor.room2.Coords.x) > 2
                || Math.Abs(currentRoom.Coords.y - corridor.room2.Coords.y) > 2)
            {
                if (deletedCorridors.Contains(corridor)) deletedCorridors.Remove(corridor);
                deletedCorridors.Add(corridor);
            }
        }
        createdCorridors.Remove(createdCorridorsGO[i].GetComponent<Corridor3D>().corridor);
        Destroy(createdCorridorsGO[i]);
        createdCorridorsGO.Remove(createdCorridorsGO[i]);
        i--;
    }
}

public void CreateDungeon(DungeonSettings settings)
{
    DungeonStructure structure = new();
    structure.rooms = CreateRooms();
    structure.roomToConnectedRooms = ConnectRooms(structure.rooms);
    structure.corridors = CreateCorridors(structure.roomToConnectedRooms);

    SaveLoadController.runInfo.dungeonStructure = structure;
}

private List<Corridor> CreateCorridors(Dictionary<Room, List<Room>>
roomToConnectedRooms)
{
    List<Corridor> corridors = new();
    List<Room> connectedRooms = new();

    foreach(var room in roomToConnectedRooms.Keys)
    {
        foreach(var connectedRoom in roomToConnectedRooms[room])
        {
            if (connectedRooms.Contains(connectedRoom)) continue;
            var newCorridor = new Corridor(room, connectedRoom);
            corridors.Add(newCorridor);
        }
        connectedRooms.Add(room);
    }
    return corridors;
}

private Dictionary<Room, List<Room>> ConnectRooms(List<Room> rooms)
{
    Dictionary<Room, List<Room>> roomToConnectedRooms = new();
    for (int i = 0; i < rooms.Count; i++)
    {
        List<Room> connectedRooms = new();
        for (int j = 0; j < rooms.Count; j++)
        {
            int xDistance = Mathf.Abs(rooms[i].Coords.x - rooms[j].Coords.x);
            int yDistance = Mathf.Abs(rooms[i].Coords.y - rooms[j].Coords.y);
            if(xDistance + yDistance == 1)
            {
                connectedRooms.Add(rooms[j]);
            }
        }
        roomToConnectedRooms[rooms[i]] = connectedRooms;
    }
    return roomToConnectedRooms;
}

private List<Room> CreateRooms()
{
    List<Room> rooms = new();
    Vector2Int nextCoords = Vector2Int.zero;

```

```

        int roomsToCreate = Random.Range(settings.minNumberOfRooms,
settings.maxNumberOfRooms);
        int countRooms = roomsToCreate;
        List<Vector2Int> usedCoords = new();

        while(roomsToCreate > 0)
        {
            if (rooms.Count > 0) nextCoords = GetNextCoords(nextCoords);
            if (usedCoords.Contains(nextCoords)) continue;
            Room room = new(rooms.Count + 1, nextCoords);
            if (room.Coords.x == 0 && room.Coords.y == 0)
            {
                string path = $"EventData/Start Events/";
                string name = $"{(int)SaveLoadController.runInfo.PlayerTeam[0].Data.charClass
- 2}-0";
                room.eventData = Resources.Load<EventData>(path + name);
                room.eventPath = path;
                room.eventName = name;
                room.roomType = Room.RoomType.StartRoom;
                SaveLoadController.runInfo.currentRoom = room;
            }
            else
            {
                //Генератор ивента будет тут
                GenerateRandomEvent(room, countRooms - 2, (countRooms - 2) - roomsToCreate);
                //int eventID = 0;
                //string path = $"EventData/";
                //string name = $"{eventID}-0";
                //room.eventData = Resources.Load<EventData>(path + name);
                //room.eventPath = path;
                //room.eventName = name;

                if (roomsToCreate == 1)
                {
                    //Также ивент встречи босса
                    room.roomType = Room.RoomType.Boss;
                    string path2 = $"EventData/Boss Events/";
                    string name2 = $"{0}-0";
                    room.eventData = Resources.Load<EventData>(path2 + name2);
                    room.eventPath = path2;
                    room.eventName = name2;
                }
            }
            rooms.Add(room);
            usedCoords.Add(nextCoords);
            roomsToCreate--;
        }
        return rooms;
    }

    public enum EventCategory
    {
        Combat,
        Good,
        Trap,
        Class
    }

    private int bias = 0; // может быть отрицательным
    private List<EventCategory> usedCategories = new();

    public EventData GenerateRandomEvent(Room room, int totalEventRooms, int
currentEventIndex)
    {
        string basePath = "EventData/";
        Dictionary<EventCategory, string> folderMap = new()
        {
            { EventCategory.Combat, "Combat Events" },
            { EventCategory.Good, "Good Events" },
            { EventCategory.Trap, "Trap Events" },

```

```

        { EventCategory.Class, $"Class
Events/{SaveLoadController.runInfo.PlayerTeam[0].Data.charClass} Events" }
    };

    // Обязательный классовой ивент
    if (currentEventIndex == totalEventRooms - 1 &&
!usedCategories.Contains(EventCategory.Class))
    {
        return GenerateSpecificCategory(EventCategory.Class, folderMap, basePath, room);
    }

    // Базовые веса
    float baseCombatWeight = 0.3f;
    float baseGoodWeight = 0.2f;
    float baseTrapWeight = 0.3f;
    float baseClassWeight = 0.2f;

    // Смещение
    float biasOffset = bias * 0.025f; //Изменение на 2.5%
    float goodWeight = Mathf.Clamp01(baseGoodWeight + biasOffset);
    float trapWeight = Mathf.Clamp01(baseTrapWeight - biasOffset);

    // Перенормируем Combat, Class
    float fixedWeight = baseCombatWeight + baseClassWeight;
    float variableWeight = goodWeight + trapWeight;
    float total = fixedWeight + variableWeight;

    Dictionary<EventCategory, float> weights = new()
    {
        { EventCategory.Combat, baseCombatWeight / total },
        { EventCategory.Good, goodWeight / total },
        { EventCategory.Trap, trapWeight / total },
        { EventCategory.Class, baseClassWeight / total }
    };

    EventCategory chosenCategory = GetRandomCategoryByWeight(weights);
    eventData chosenEvent = GenerateSpecificCategory(chosenCategory, folderMap,
basePath, room);

    // Обновляем bias
    if (chosenCategory == EventCategory.Combat || chosenCategory == EventCategory.Trap)
        bias--;
    else if (chosenCategory == EventCategory.Good)
        bias++;

    return chosenEvent;
}

private eventData GenerateSpecificCategory(EventCategory category,
Dictionary<EventCategory, string> folderMap, string basePath, Room room)
{
    string folderPath = basePath + folderMap[category];
    List<eventData> events = new(Resources.LoadAll<eventData>(folderPath));
    events.RemoveAll(e => e.eventID_Part != 0);

    if (events.Count == 0)
    {
        Debug.LogWarning($"No usable events in category {category} at {folderPath}");
        return null;
    }

    eventData chosen = events[UnityEngine.Random.Range(0, events.Count)];
    room.eventData = chosen;
    room.eventPath = folderPath + "/";
    room.eventName = chosen.name;
    room.roomType = Room.RoomType.Common;

    if (!usedCategories.Contains(category))
        usedCategories.Add(category);
    return chosen;
}

```

```

private EventCategory GetRandomCategoryByWeight(Dictionary<EventCategory, float>
weights)
{
    float roll = UnityEngine.Random.value;
    float cumulative = 0f;

    foreach (var pair in weights)
    {
        cumulative += pair.Value;
        if (roll <= cumulative)
            return pair.Key;
    }

    return EventCategory.Combat;
}

private Vector2Int GetNextCoords(Vector2Int nextCoords)
{
    float randomNumber = Random.Range(0, 4);
    switch (randomNumber)
    {
        case 0:
            return new Vector2Int(nextCoords.x - 1, nextCoords.y);
        case 1:
            return new Vector2Int(nextCoords.x + 1, nextCoords.y);
        case 2:
            return new Vector2Int(nextCoords.x, nextCoords.y - 1);
        default:
            return new Vector2Int(nextCoords.x, nextCoords.y + 1);
    }
}
}

```

ПРИЛОЖЕНИЕ В. ФАЙЛ FIGHT.CS

Файл `Fight.cs` – основной скрипт, реализующий цикл боя. Большинство переменных, связанных с боем, хранятся здесь. Также реализуются такие функции как: начало боя, ход игрока и ход противника, передача хода, выбор умения противником для атаки, проверка на конец раунда и начало нового, а также окончание сражения.

```
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

public class Fight : MonoBehaviour
{
    public GridLayoutGroup heroes, enemies;

    public CanvasGroup playerHand;

    public static List<Fighter> AllCharacter
    {
        get
        {
            var list = new List<Fighter>(EnemyTeam);
            foreach (Fighter hero in PlayerTeam)
                list.Add(hero);
            return list;
        }
    }

    public static List<PlayableCharacter> PlayerTeam;
    public static List<Fighter> PlayerUITeam;

    public static List<Fighter> EnemyTeam;
    public static List<Fighter> EnemyUITeam;

    public static List<Fighter> AlreadyTurn;

    public int round_number = 1;

    public List<SpriteRenderer> HeroesSprites;
    public List<SpriteRenderer> EnemiesSprites;
    public List<SpriteRenderer> AddictSprites;

    public List<GameObject> SkillsCards;

    public GameObject skipBtn;

    public GameObject WinLosePanel;
    public TMP_Text WinLoseText;
    public TMP_Text RoundNum;

    private static int SelectedCharacterID = -1;
    private static bool _selected = false;
    public static Skill selectedSkill = null;
    public static Fighter selectedTarget = null;

    public static bool needUpdatePortrait = false;

    public static bool cast = false;
    public static bool seeIntension = false;
```

```

private bool allEnemiesDoTurn, allPlayerCharactersDoTurn;
public static bool isWin, isLose, isAllDoTurn;
public static bool endFight = false;
public static bool isEnemyTurn = false;
public static bool skipTurn = false;
public static bool isNotOver = false;

public static List<CastSkillStructure> additionalCastSkills = new();

public static Room eventRoom; //Откуда запустилась битва

public static readonly int procentPerOneCharacteristic = 5; // Для проверок влияния хар-к
public static readonly int limitProcent = 80;
private void Start()
{
    Cursor.lockState = CursorLockMode.None;

    playerHand.alpha = 1f;
    playerHand.interactable = true;
    playerHand.blocksRaycasts = true;

    SelectedCharacterID = -1;
    _selected = false;
    selectedSkill = null;
    selectedTarget = null;
    needUpdatePortrait = false;
    cast = false;
    isEnemyTurn = false;
    endFight = false;
    skipTurn = false;
    isNotOver = false;
    additionalCastSkills = new();

    PlayerTeam = SaveLoadController.runInfo.PlayerTeam;
    EnemyTeam = SaveLoadController.enemies;

    //Summon Buffs
    for (int i = 0; i < PlayerTeam.Count; i++)
    {
        var character = PlayerTeam[i];
        foreach (var skill in character.skills)
        {
            var data = skill.skillData;
            if(data.skill_type == SkillSO.SkillType.Summon &&
                data.skill_target == SkillSO.SkillTarget.Passive)
            {
                switch (data.passiveBuff)
                {
                    case Fighter.Buff.ScreamIntoVoid:
                        //Призываем 3 духов разного типа стартовых персонажей
                        //Сумма их статов = сумме параметров жрицы, распределённых с упором на
главные характеристики
                        //Когда сумма параметров Жрицы достигает определённого порога, они
получают новые активные навыки из сета персонажа (случайно)
                        int count = 3;
                        int sumParam = character.strength + character.agility + character.wisdom
                            + character.constitution + character.defence;

                        // Разделяем общее количество параметров между духами
                        int[] statsPerSpirit = DistributeStats(sumParam, count);

                        for (int k = 0; k < count; k++)
                        {
                            var list = new
List<CharacterSO>(Resources.LoadAll<CharacterSO>("CharData/Playable/"));
                            list.Remove(character.Data);
                            var randomClass = list[Random.Range(0, list.Count)];
                            var ally = new PlayableCharacter(randomClass.name)
                            {
                                isSummon = true
                            };

                            int spiritStats = statsPerSpirit[k];
                            var stats = new Dictionary<CharacterSO.Stat, int>
                            {

```

```

        { CharacterSO.Stat.Strength, 0 },
        { CharacterSO.Stat.Agility, 0 },
        { CharacterSO.Stat.Wisdom, 0 },
        { CharacterSO.Stat.Constitution, 1 }, // Минимум 1
        { CharacterSO.Stat.Defence, 0 }
    };
    int remaining = spiritStats - 1; // уже потратили 1 на constitution

    var priorities = randomClass.priorityStats;
    int priorityShare = Mathf.RoundToInt(remaining * 0.75f);
    int nonPriorityShare = remaining - priorityShare;

    // Распределение приоритетных очков
    for (int j = 0; j < priorityShare; j++)
    {
        CharacterSO.Stat stat = priorities[Random.Range(0,
priorities.Count)];
        stats[stat]++;
    }

    // Распределение оставшихся очков
    var nonPriorities = stats.Keys.Except(priorities).ToList();
    for (int j = 0; j < nonPriorityShare; j++)
    {
        CharacterSO.Stat stat = nonPriorities[Random.Range(0,
nonPriorities.Count)];
        stats[stat]++;
    }

    // Присваиваем статы
    ally.strength = stats[CharacterSO.Stat.Strength];
    ally.agility = stats[CharacterSO.Stat.Agility];
    ally.wisdom = stats[CharacterSO.Stat.Wisdom];
    ally.constitution = stats[CharacterSO.Stat.Constitution];
    ally.defence = stats[CharacterSO.Stat.Defence];

    ally.FullHeal();

    while(ally.skills.Count != 1 && sumParam < 25)
        ally.skills.RemoveAt(1);

    if (sumParam >= 50)
    {
        var skillPools = ally.AvailableSkills;

        int skillsToAdd = (sumParam - 50) / 25 + 1; // Например: 50 → 1, 75
→ 2, 100 → 3

        List<SkillSO> availableSkills = new();
        foreach(SkillPool pool in skillPools)
        {
            foreach(SkillSO skillSO in pool.activeSkillList)
            {
                if (!availableSkills.Contains(skillSO))
                {
                    availableSkills.Add(skillSO);
                }
            }
        }

        availableSkills.Remove(ally.skills[1].skillData);

        for (int j = 0; j < skillsToAdd && availableSkills.Count > 0; j++)
        {
            var skill1 = availableSkills[Random.Range(0,
availableSkills.Count)];
            ally.AddSkill(skill1);
            availableSkills.Remove(skill1); // Чтобы не повторялись
        }

        ally.AddSkill("Corpseless");

        if (PlayerTeam.Count < 6)

```

```

        PlayerTeam.Add(ally);
    }
    break;
case Fighter.Buff.WanderingMusician:
    Fighter summon = new("Summons/Accordion Accompanist");
    summon.FullHeal();
    if (PlayerTeam.Count < 6)
        PlayerTeam.Add(summon as PlayableCharacter);
    break;
case Fighter.Buff.AngelicGuardian:
    summon = new("Summons/Angel");
    summon.FullHeal();
    if (PlayerTeam.Count < 6)
        PlayerTeam.Add(summon as PlayableCharacter);
    break;
    }
}
}

PlayerUITeam = new List<Fighter>(PlayerTeam);
EnemyUITeam = new List<Fighter>(EnemyTeam);

SelectedCharacterID_Reset();

AlreadyTurn = new List<Fighter>();
//AllCharacter = new List<Fighter>();
//foreach (Fighter character in PlayerTeam) AllCharacter.Add(character);
//foreach (Fighter character in EnemyTeam) AllCharacter.Add(character);

//Подготовка персонажей
foreach (Fighter character in AllCharacter)
{
    character.armor = character.defence * 2;
    character.bonus_hp = 0;
    character.bonus_strength = 0;
    character.bonus_wisdom = 0;
    character.bonus_agility = 0;
    character.buffs = new();
}

FightUIController.hardUpdate = true;

FightStart();
}

private void Update()
{
    //Возврат по правой кнопки мыши
    if (Input.GetKeyDown(KeyCode.Mouse1) && selectedSkill == null && !isEnemyTurn
        && !BuffPanelController.isOpened && PlayerTeam.Count != 1)
    {
        CardShowerReset();
        SelectedCharacterID_Reset();
        _selected = false;
        PlayerUITeam = new List<Fighter>(PlayerTeam);
        UpdatePortrait();
        FightUIController.hardUpdate = true;
        skipBtn.SetActive(false);
    }
    else if (Input.GetKeyDown(KeyCode.Mouse1) && !isEnemyTurn &&
        !BuffPanelController.isOpened && selectedSkill != null)
    {
        selectedSkill = null;
        EnemyUITeam = new List<Fighter>(EnemyTeam);
        UpdatePortrait();
        FightUIController.hardUpdate = true;
    }

    //Вывод выбранного персонажа
    if (SelectedCharacterID != -1 && !_selected)
    {
        _selected = true;
        PlayerUITeam = new List<Fighter> { PlayerTeam[SelectedCharacterID] };
    }
}

```



```

        UpdatePortrait();
        if(PlayerTeam.Count != 1)
            skipBtn.SetActive(true);
    }

    //Обновление по требованию
    if (needUpdatePortrait)
    {
        needUpdatePortrait = false;
        UpdatePortrait();
    }
    RoundNum.text = "Раунд: " + round_number + (seeIntension ? " (намерения видны)" : "");
}

int[] DistributeStats(int total, int count)
{
    int[] result = new int[count];
    int remaining = total;

    for (int i = 0; i < count; i++)
    {
        result[i] = total / count;
        remaining -= result[i];
    }

    while (remaining > 0)
    {
        int index = Random.Range(0, count);
        result[index]++;
        remaining--;
    }

    return result;
}

public void CardShowerReset()
{
    foreach (var SkCard in SkillsCards)
    {
        SkCard.GetComponent<CardShower>().isFirstMove = false;
        SkCard.GetComponent<CardShower>().isLock = false;
    }
    Skill_Image.isOneLocked = false;
    Skill_Image.isNeedClose = true;
}

private void FightStart()
{
    foreach (var chara in AllCharacter)
        chara.PrepareToFight();
    foreach (Fighter character in AllCharacter)
    {
        character.isSpawn = false;
        character.Spawn();
    }

    UpdatePortrait();
    //MakeIntention();

    //StartFight Buffs
    foreach (Fighter character in AllCharacter)
    {
        foreach (var buff in character.buffs)
        {
            switch (buff)
            {
                case Fighter.Buff.QuickRebuff:
                    character.CastSkill(new List<Fighter>() { RandomEnemy(character) },
SkillDB.Instance.GetSkillByName("Basic Attack"), false);
                    break;
                case Fighter.Buff.EchoHope:
                    int heal = (int)(character.max_hp * 0.15f);
                    if (heal == 0) heal = 1;
                    character.TakeHeal(character, heal);
                    break;
            }
        }
    }
}

```

```

        case Fighter.Buff.NoAttack:
            var skill = SkillDB.Instance.GetSkillByName("Basic Attack");
            if (character.skillsBattle.Contains(skill))
            {
                character.skillsBattle.Remove(skill);
            }
            break;
    }
}

StartRound();
}

public void StartRound()
{
    //Проверка на просмотр намерений:
    var mainChar = PlayerTeam[0];
    int chance = (mainChar.wisdom + mainChar.bonus_wisdom) * procentPerOneCharacteristic;
    if (chance > limitProcent) chance = limitProcent;
    int res = Random.Range(0, 100);
    seeIntension = res < chance;

    foreach (var enemy in EnemyTeam)
    {
        //Проверка на страх:
        chance = (mainChar.strength + mainChar.bonus_strength) - (enemy.strength +
enemy.bonus_strength) * procentPerOneCharacteristic;
        if (chance > limitProcent) chance = limitProcent;
        res = Random.Range(0, 100);
        enemy.isFear = res < chance;
        if (res < chance)
        {
            AlreadyTurn.Add(enemy);
        }
    }

    //Всё что нужно делать в начале раунда из баффов
    foreach (var chara in AllCharacter)
    {
        foreach (var buff in chara.buffs)
        {
            switch (buff)
            {
                case Fighter.Buff.Accompaniment:
                    List<Fighter> list = new(EnemyTeam);
                    if (!EnemyTeam.Contains(chara))
                    {
                        list = new(PlayerTeam);
                    }
                    bool contains = false;
                    foreach (var figh in list)
                    {
                        if (figh.Data.name == "Ancient Evil")
                        {
                            contains = true;
                            break;
                        }
                    }
                    if (!contains)
                        chara.buffs.Remove(Fighter.Buff.Accompaniment);
                    break;
                case Fighter.Buff.WeightMemories:
                    chance = 30;
                    res = Random.Range(0, 100);
                    chara.isFear = res < chance;
                    if (res < chance)
                    {
                        AlreadyTurn.Add(chara);
                    }
                    break;
                case Fighter.Buff.ScarBetrayal:
                    chara.SacrificeHP(1);
                    break;
            }
        }
    }
}

```

```

    }
}
CheckConditionWinLose();

PlayerUITeam = new(PlayerTeam);
EnemyUITeam = new(EnemyTeam);
UpdatePortrait();
FightUIController.hardUpdate = true;

// Кто стартует раунд?
bool isPlayerFirst = true;
int playerAgility = PlayerTeam[0].agility + PlayerTeam[0].bonus_agility;
int maxEnemyAgility = int.MinValue;

foreach (var chara in EnemyTeam)
{
    int enemyAgility = chara.agility + chara.bonus_agility;
    if (enemyAgility > maxEnemyAgility)
        maxEnemyAgility = enemyAgility;
}

if (maxEnemyAgility > playerAgility)
{
    isPlayerFirst = false;
}
else if (maxEnemyAgility == playerAgility)
{
    isPlayerFirst = Random.value < 0.5f;
}

if (isPlayerFirst)
    StartCoroutine(PlayerTurn());
else
    StartCoroutine(EnemyTurn());
}

public void SelectedCharacterID_Reset()
{
    SelectedCharacterID = -1;
    if (PlayerTeam.Count == 1)
        SelectedCharacterID = 0;
}

IEnumerator PlayerTurn()
{
    if (allPlayerCharactersDoTurn) goto Skip;
    SelectedCharacterID_Reset();
    StartTurn:
    skipBtn.SetActive(false);
    selectedSkill = null;
    selectedTarget = null;
    CardShowerReset();

    if (skipTurn)
        goto SkipTurn;

    //Ждём пока не выберется скилл
    ChangeSkill:
    EnemyUITeam = new List<Fighter>(EnemyTeam);
    UpdatePortrait();
    if (PlayerTeam.Count == 1)
        skipBtn.SetActive(true);

    while (selectedSkill == null)
    {
        yield return null;
        if (skipTurn)
        {
            goto StartTurn;
        }
    }

    //Кто отображается в списке персонажей справа
    PlayerUITeam = new List<Fighter>() { SelectedCharacter() };
}

```

```

switch (selectedSkill.skillData.skill_target)
{
    case SkillSO.SkillTarget.Solo_Enemy:
    case SkillSO.SkillTarget.Mass_Enemies:
    case SkillSO.SkillTarget.Random_Enemy:
        EnemyUITeam = new List<Fighter>(EnemyTeam);
        break;
    case SkillSO.SkillTarget.All:
    case SkillSO.SkillTarget.Random_Target:
        var list = new List<Fighter>(AllCharacter);
        list.Remove(SelectedCharacter());
        EnemyUITeam = list;
        break;
    case SkillSO.SkillTarget.Solo_Ally:
    case SkillSO.SkillTarget.Mass_Allies:
    case SkillSO.SkillTarget.Random_Ally:
        EnemyUITeam = new List<Fighter>(PlayerTeam);
        break;
    case SkillSO.SkillTarget.Caster:
        EnemyUITeam = new List<Fighter>() { SelectedCharacter() };
        break;
}
UpdatePortrait();
FightUIController.allInteractable = true;
FightUIController.hardUpdate = true;

var temp_skill = selectedSkill;

//Ждём выбора цели
while (selectedTarget == null)
{
    yield return null;
    if (Input.GetKeyDown(KeyCode.Mouse1))
    {
        FightUIController.allInteractable = false;
        _selected = false;
        goto StartTurn;
    }

    if(temp_skill != selectedSkill)
    {
        FightUIController.allInteractable = false;
        goto ChangeSkill;
    }

    if (skipTurn)
    {
        FightUIController.allInteractable = false;
        _selected = false;
        goto StartTurn;
    }
}
//Выбираем цели для каста
var selectedTargets = ChooseTarget(selectedSkill, SelectedCharacter(), selectedTarget);

FightUIController.allInteractable = false;
_selected = false;

PlayerUITeam = new List<Fighter> { PlayerTeam[SelectedCharacterID] };
EnemyUITeam = selectedTargets;
cast = true;
UpdatePortrait();
FightUIController.hardUpdate = true;

SelectedCharacter().CastSkill(selectedTargets, selectedSkill);

SkipTurn:

AlreadyTurn.Add(SelectedCharacter());

skipBtn.SetActive(false);

if (!skipTurn)
    yield return new WaitForSeconds(2f);

```

```

//Если применение скилла повлекло за собой применение других скиллов
while (additionalCastSkills.Count > 0)
{
    //Устанавливаем весь дизайн для каста и кастуем здесь
    selectedTargets = additionalCastSkills[0].targets;
    selectedSkill = additionalCastSkills[0].skill;

    Skill_Image.externalSkill = selectedSkill;
    Skill_Image.externalCaster = SelectedCharacter();

    PlayerUITeam = new List<Fighter> { PlayerTeam[SelectedCharacterID] };
    EnemyUITeam = selectedTargets;
    cast = true;
    UpdatePortrait();
    FightUIController.hardUpdate = true;

    SelectedCharacter().CastSkill(selectedTargets, selectedSkill,
additionalCastSkills[0].needCooldown);
    additionalCastSkills.RemoveAt(0);
    yield return new WaitForSeconds(2f);
}

PlayerUITeam = new List<Fighter>(PlayerTeam);
EnemyUITeam = new List<Fighter>(EnemyTeam);

selectedSkill = null;
selectedTarget = null;
SelectedCharacterID_Reset();
_selected = false;
skipTurn = false;

cast = false;
UpdatePortrait();

Skip:
CardShowerReset();
CheckRoundChange(SelectedCharacter());
if (!isWin && !isLose)
{
    if (isAllDoTurn) StartRound();
    else StartCoroutine(EnemyTurn());
}
}

IEnumerator EnemyTurn()
{
    Fighter caster = null;
    if (allEnemiesDoTurn) goto Skip;
    var notTurnEnemies = new List<Fighter>();
    foreach (Fighter figh in EnemyTeam)
    {
        if (!AlreadyTurn.Contains(figh) && !figh.isDead) notTurnEnemies.Add(figh);
    }
    if (notTurnEnemies.Count == 0) goto Skip;

    isEnemyTurn = true;
    UpdatePortrait();
    FightUIController.allDisable = true;
    FightUIController.hardUpdate = true;

    yield return new WaitForSeconds(2f);

    caster = notTurnEnemies[Random.Range(0, notTurnEnemies.Count)];

    if (caster.Intension == null)
        goto SkipTurn;

    //EnemyUITeam = new List<Fighter>() { selectedEnemy };
    //UpdatePortrait();
    //Реализовать логику выбора цели
    Fighter target;
    var possibleTargets = PlayerTeam
.Where(f => !f.isDead && !f.buffs.Contains(Fighter.Buff.Provocation))
.ToList();
    if (possibleTargets.Count > 0)

```

```

        target = possibleTargets[Random.Range(0, possibleTargets.Count)];
    else
        target = PlayerTeam.First(f => !f.isDead); // fallback

    var selectedTargets = ChooseTarget(caster.Intension, caster, target);

    Debug.Log("Использует умение: " + caster.Intension.skillData._name);
    PlayerUITeam = selectedTargets;
    EnemyUITeam = new List<Fighter> { caster };
    cast = true;
    Skill_Image.externalSkill = caster.Intension;
    Skill_Image.externalCaster = caster;
    UpdatePortrait();

    yield return new WaitForSeconds(2f);

    caster.CastSkill(selectedTargets, caster.Intension);

    SkipTurn:
    AlreadyTurn.Add(caster);

    //Если применение скилла повлекло за собой применение других скиллов
    while (additionalCastSkills.Count > 0)
    {
        //Устанавливаем весь дизайн для каста и кастуем здесь
        selectedTargets = additionalCastSkills[0].targets;
        var selectedSkill = additionalCastSkills[0].skill;

        Skill_Image.externalSkill = selectedSkill;
        Skill_Image.externalCaster = caster;

        PlayerUITeam = selectedTargets;
        EnemyUITeam = new List<Fighter> { caster };
        cast = true;
        UpdatePortrait();
        FightUIController.hardUpdate = true;

        yield return new WaitForSeconds(2f);
        caster.CastSkill(selectedTargets, selectedSkill,
additionalCastSkills[0].needCooldown);
        additionalCastSkills.RemoveAt(0);
    }
    yield return null;
    cast = false;
    Skill_Image.isDisableExSkill = true;
    yield return null;
    PlayerUITeam = new List<Fighter>(PlayerTeam);
    EnemyUITeam = new List<Fighter>(EnemyTeam);
    FightUIController.allDisable = false;

    UpdatePortrait();
    isEnemyTurn = false;
    Skip:
    CheckRoundChange(caster);
    if (!isWin && !isLose)
    {
        if (isAllDoTurn) StartRound();
        else StartCoroutine(PlayerTurn());
    }
}

bool isDoubleTurn = false;

private void CheckRoundChange(Fighter whoMakeTurn)
{
    for (int i = 0; i < EnemyTeam.Count; i++)
    {
        Fighter chara = EnemyTeam[i];
        if (chara.isDead && chara.buffs.Contains(Fighter.Buff.Corpseless))
            EnemyTeam.Remove(chara);
    }

    if (isDoubleTurn && whoMakeTurn == PlayerTeam[0])
    {
        PlayerTeam[0].isDoubleTurn = false;
    }
}

```

```

    }

    if (!isDoubleTurn && whoMakeTurn == PlayerTeam[0])
    {
        //Проверка на двойной ход:
        var mainChar = PlayerTeam[0];
        int chance = (mainChar.agility + mainChar.bonus_agility) *
procentPerOneCharacteristic;
        if (chance > limitProcent) chance = limitProcent;
        int res = Random.Range(0, 100);
        PlayerTeam[0].isDoubleTurn = res < chance;
        if (res < chance)
        {
            AlreadyTurn.Remove(mainChar);
            //Кулдауны
            var keys = new List<Skill>(mainChar.cooldowns.Keys);
            for (int i = 0; i < keys.Count; i++)
            {
                Skill skill = keys[i];
                mainChar.cooldowns[skill]--;

                if (mainChar.cooldowns[skill] == 0)
                {
                    mainChar.cooldowns.Remove(skill);
                }
            }
            isDoubleTurn = true;
        }
    }

    PlayerUITeam = new(PlayerTeam);
    EnemyUITeam = new(EnemyTeam);
    UpdatePortrait();
    FightUIController.hardUpdate = true;

    CheckConditionWinLose();

    isAllDoTurn = allEnemiesDoTurn && allPlayerCharactersDoTurn;

    if (isAllDoTurn)
    {
        //Всё что нужно делать в конце раунда из баффов, начало раунда (если будет нужно) в
методе StartRound();
        foreach (var chara in AllCharacter)
        {
            foreach(var buff in chara.buffs)
            {
                switch (buff)
                {
                    case Fighter.Buff.CurseVictim:
                        chara.SacrificeHP(1);
                        break;
                    case Fighter.Buff.CurseDestruction:
                        chara.TakeDmg(null, 1, SkillSO.SkillElement.Dark);
                        break;
                }
            }

            if (chara.buffs.Contains(Fighter.Buff.Poison))
            {
                chara.TakeDmg(null, chara.poisonStacks, SkillSO.SkillElement.Poison);
                chara.poisonStacks /= 2;
                if(chara.poisonStacks <= 0)
                {
                    chara.poisonStacks = 0;
                    chara.buffs.Remove(Fighter.Buff.Poison);
                }
            }
        }

        AlreadyTurn.Clear();
        allEnemiesDoTurn = false;
        allPlayerCharactersDoTurn = false;
    }
}

```

```

isDoubleTurn = false;
PlayerTeam[0].isDoubleTurn = false;
round_number++;
Debug.Log("Следующий раунд: " + round_number.ToString());

//Кулдауны
foreach (var hero in AllCharacter)
{
    var keys = new List<Skill>(hero.cooldowns.Keys);
    for (int i = 0; i < keys.Count; i++)
    {
        Skill skill = keys[i];
        hero.cooldowns[skill]--;

        if (hero.cooldowns[skill] == 0)
        {
            hero.cooldowns.Remove(skill);
        }
    }
}

//Намерения
foreach (var enemy in EnemyTeam)
    enemy.MakeIntention();

CheckConditionWinLose();
}

private void CheckConditionWinLose()
{
    //Проверка надо ли удалять трупы
    for (int i = 0; i < PlayerTeam.Count; i++)
    {
        PlayableCharacter chara = PlayerTeam[i];
        if (chara.isDead && chara.buffs.Contains(Fighter.Buff.Corpseless))
            PlayerTeam.Remove(chara);
    }

    allPlayerCharactersDoTurn = true;
    foreach (Fighter fighter in PlayerTeam)
        if (!AlreadyTurn.Contains(fighter) && !fighter.isDead)
        {
            allPlayerCharactersDoTurn = false;
            break;
        }

    allEnemiesDoTurn = true;
    foreach (Fighter fighter in EnemyTeam)
        if (!AlreadyTurn.Contains(fighter) && !fighter.isDead)
        {
            allEnemiesDoTurn = false;
            break;
        }

    //Check WinLose
    isWin = true;
    foreach (Fighter fighter in EnemyTeam)
        if (!fighter.isDead)
        {
            isWin = false;
            break;
        }

    isLose = true;
    foreach (Fighter fighter in PlayerTeam)
        if (!fighter.isDead)
        {
            isLose = false;
            break;
        }

    if (isLose || isWin)
    {
        EndFight();
        return;
    }
}

```



```

private void EndFight()
{
    StopAllCoroutines();
    if (endFight) return;
    endFight = true;
    Skill_Image.isNeedClose = true;
    WinLosePanel.SetActive(true);

    playerHand.alpha = 0f;
    playerHand.interactable = false;
    playerHand.blocksRaycasts = false;

    if (isLose)
    {
        WinLoseText.text = "К сожалению, вы проиграли";
        SaveLoadController.EndFight();
        var data = eventRoom.eventData;
        if (data.isNotOver)
        {
            WinLoseText.text += "\nНо это ещё не конец.";
            eventRoom.eventData = eventRoom.eventData.choices[1];
            var data2 = eventRoom.eventData;
            eventRoom.eventName = $"{data2.eventID}-{data2.eventID_Part}";

            foreach(var chara in PlayerTeam)
            {
                chara.hp = (int)(chara.max_hp * (data.overHpProcent / 100.0f));
            }
            SaveLoadController.Save();
            isNotOver = true;
        }
        else
            SaveLoadController.ClearSave(SaveLoadController.slot);
    }
    else if (isWin)
    {
        WinLoseText.text = "Поздравляю с победой";

        eventRoom.eventData = eventRoom.eventData.choices[0];
        var data = eventRoom.eventData;
        eventRoom.eventName = $"{data.eventID}-{data.eventID_Part}";
        SaveLoadController.Save();
    }
}

public static void SelectCharacter(int id)
{
    SelectedCharacterID = id;
}

public static PlayableCharacter SelectedCharacter()
{
    if (SelectedCharacterID == -1) return null;
    return PlayerTeam[SelectedCharacterID];
}

public static void SelectTarget(int id)
{
    selectedTarget = id < 6 ? PlayerTeam[id] : EnemyTeam[id - 6];
}

public static Fighter RandomEnemy(Fighter enemyForMe)
{
    List<Fighter> enemies = new(EnemyTeam);
    if (EnemyTeam.Contains(enemyForMe))
    {
        enemies = new(PlayerTeam);
    }
    for (int i = 0; i < enemies.Count; i++)
    {
        if(enemies[i].isDead)
        {
            enemies.RemoveAt(i);
            i--;
        }
    }
    if (enemies.Count == 0) return EnemyTeam[0];
}

```

```

        return enemies[Random.Range(0, enemies.Count)];
    }

    public static bool IsEnemy(Fighter me, Fighter him)
    {
        return !(PlayerTeam.Contains(me) && PlayerTeam.Contains(him)
            ||
            EnemyTeam.Contains(me) && EnemyTeam.Contains(him));
    }

    public static List<Fighter> ChooseTarget(Skill selectedSkill, Fighter caster, Fighter
selectedTarget)
    {
        List<Fighter> allies = new(PlayerTeam);
        List<Fighter> enemies = new(EnemyTeam);
        if(EnemyTeam.Contains(caster))
        {
            allies = new(EnemyTeam);
            enemies = new(PlayerTeam);
        }
        List<Fighter> selectedTargets = new();
        switch (selectedSkill.skillData.skill_target)
        {
            case SkillSO.SkillTarget.Solo_Enemy:
            case SkillSO.SkillTarget.Solo_Ally:
                selectedTargets.Add(selectedTarget);
                break;
            case SkillSO.SkillTarget.Caster:
                selectedTargets.Add(caster);
                break;
            case SkillSO.SkillTarget.Mass_Enemies:
                selectedTargets = new List<Fighter>(enemies);
                if (!selectedSkill.skillData.isCorpseTargetToo)
                {
                    for (int i = 0; i < selectedTargets.Count; i++)
                    {
                        if (selectedTargets[i].isDead)
                        {
                            selectedTargets.Remove(selectedTargets[i]);
                            i--;
                        }
                    }
                }
                break;
            case SkillSO.SkillTarget.All:
                selectedTargets = new List<Fighter>(AllCharacter);
                selectedTargets.Remove(caster);
                if (!selectedSkill.skillData.isCorpseTargetToo)
                {
                    for (int i = 0; i < selectedTargets.Count; i++)
                    {
                        if (selectedTargets[i].isDead)
                        {
                            selectedTargets.Remove(selectedTargets[i]);
                            i--;
                        }
                    }
                }
                break;
            case SkillSO.SkillTarget.Random_Enemy:
                selectedTargets = new List<Fighter>() { enemies[Random.Range(0, enemies.Count)] };
                if (!selectedSkill.skillData.isCorpseTargetToo)
                {
                    do
                    {
                        selectedTargets = new List<Fighter>() { enemies[Random.Range(0,
enemies.Count)] };
                    } while (selectedTargets[0].isDead);
                }
                break;
            case SkillSO.SkillTarget.Mass_Allies:
                selectedTargets = new List<Fighter>(allies);
                if (!selectedSkill.skillData.isCorpseTargetToo)
                {
                    for (int i = 0; i < selectedTargets.Count; i++)

```

```

        {
            if (selectedTargets[i].isDead)
            {
                selectedTargets.Remove(selectedTargets[i]);
                i--;
            }
        }
    }
    break;
case SkillSO.SkillTarget.Random_Ally:
    selectedTargets = new List<Fighter>() { allies[Random.Range(0, allies.Count)] };
    if (!selectedSkill.skillData.isCorpseTargetToo)
    {
        do
        {
            selectedTargets = new List<Fighter>() { allies[Random.Range(0,
allies.Count)] };
        } while (selectedTargets[0].isDead);
    }
    break;
case SkillSO.SkillTarget.Random_Target:
    List<Fighter> list = new List<Fighter>();
    list.Remove(caster);
    selectedTargets = new List<Fighter>() { list[Random.Range(0, list.Count)] };
    if (!selectedSkill.skillData.isCorpseTargetToo)
    {
        do
        {
            selectedTargets = new List<Fighter>() { list[Random.Range(0, list.Count)] };
        } while (selectedTargets[0].isDead);
    }
    break;
}
return selectedTargets;
}

private void UpdatePortrait()
{
    FightUIController.CountHeroes = PlayerUITeam.Count;
    FightUIController.CountEnemies = EnemyUITeam.Count;

    for (int i = 0; i < PlayerUITeam.Count; i++)
    {
        if(i > 5)
            AddictSprites[i - 6].sprite = PlayerUITeam[i].Portrait;
        else
            HeroesSprites[i].sprite = PlayerUITeam[i].Portrait;
    }
    for (int i = 0; i < EnemyUITeam.Count; i++)
    {
        if (i > 5)
            AddictSprites[i - 6].sprite = EnemyUITeam[i].Portrait;
        else
            EnemiesSprites[i].sprite = EnemyUITeam[i].Portrait;
    }
    if(PlayerUITeam.Count == 1)
    {
        int i = 0;
        if(!isEnemyTurn)
        {
            foreach (Skill skill in PlayerUITeam[0].skillsBattle)
            {
                if (skill.skillData.skill_target == SkillSO.SkillTarget.Passive) continue;
                var SkCard = SkillsCards[i];
                SkCard.GetComponent<Graphic>().enabled = true;
                Show(SkCard);
                SkCard.GetComponent<CardFiller>().skill = skill;
                SkCard.GetComponent<Skill_Image>().skill = skill;

                var contain = PlayerUITeam[0].cooldowns.Keys.Contains(skill);
                SkCard.GetComponent<Image>().raycastTarget = !contain;
                SkCard.transform.GetChild(2).gameObject.SetActive(contain);
                if (contain)
                {

```

```

SkCard.transform.GetChild(2).GetChild(1).GetComponent<TextMeshProUGUI>().text =
PlayerUITeam[0].cooldowns[skill].ToString();
    }
    //SkIm.sprite = skill.Icon;
    //SkIm.gameObject.GetComponent<Skill_Image>().skill = skill;
    i++;
}
for(;i< SkillsCards.Count; i++)
{
    Hide(SkillsCards[i]);
}
}
else
{
    AllHide();
}
}
else
{
    AllHide();
}
}

void AllHide()
{
    foreach (var SkCard in SkillsCards)
    {
        Hide(SkCard);
    }
}

void Show(GameObject go)
{
    go.GetComponent<LayoutElement>().ignoreLayout = false;
    go.GetComponent<Graphic>().enabled = true;
    ShowChildren(go);
}

void Hide(GameObject go)
{
    go.GetComponent<LayoutElement>().ignoreLayout = true;
    HideChildren(go);
    go.GetComponent<Graphic>().enabled = false;
}

void HideChildren(GameObject go)
{
    Graphic[] lChildRenderers = go.GetComponentsInChildren<Graphic>();
    foreach (Graphic lRenderer in lChildRenderers)
    {
        lRenderer.enabled = false;
    }
}

void ShowChildren(GameObject go)
{
    Graphic[] lChildRenderers = go.GetComponentsInChildren<Graphic>();
    foreach (Graphic lRenderer in lChildRenderers)
    {
        lRenderer.enabled = true;
    }
}

public void SkipBtn()
{
    skipTurn = true;
}
}

```

ПРИЛОЖЕНИЕ Г. ФАЙЛ FIGHTER.CS

Файл `Fighter.cs` – представляет собой основной класс-описание боевого юнита (персонажа или противника), участвующего в пошаговых сражениях. Данный класс хранит характеристики бойца, активные и пассивные умения, текущие значения здоровья, брони, яда, состояния и другие параметры, влияющие на поведение бойца в бою.

```
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using UnityEngine;
using UnityEngine.TextCore.Text;
using static UnityEngine.GraphicsBuffer;

public class Fighter
{
    //Данные
    public CharacterSO Data;
    //public int lvl = 1;

    //Боевые характеристики
    public int hp;
    public int max_hp
    {
        get
        {
            if (constitution <= 0)
                return 1;
            return constitution * 5;
        }
    }
    public int bonus_hp; //если в бою чьи-то хп повысили или понизили

    //Сохраняемые эффекты (между боями)
    public Dictionary<Effect, int> effectStacks = new();
    public int poisonStacks = 0;

    public int defence; //дополнительное здоровье, которое может превышать максимум
    public int armor; //дополнительное здоровье, которое может превышать максимум
    public Skill Intension = null; //Намерение использовать скилл
    public Skill prevIntension = null;

    public bool isDead = false;

    //Характеристики
    public int strength, agility, wisdom, constitution;
    public int bonus_strength, bonus_agility, bonus_wisdom; //боевые изменения хар-тик

    public List<Buff> buffs = new();

    public Dictionary<Skill, short> cooldowns = new();

    public bool isSpawn;
    public bool isSummon;
    public bool isFear;

    //Умения персонажа
    public List<Skill> skills = new()
    {
        SkillDB.Instance.GetSkillByName("Basic Attack")
    };
    public List<Skill> skillsBattle; //в бою используем этот список, на случай если изменится что-нибудь
```

```

public Fighter(string name)
{
    Data = Resources.Load<CharacterSO>("CharData/" + name);
    Fill(Data);
}
public Fighter(CharacterSO data)
{
    Data = data;
    Fill(Data);
}

private void Fill(CharacterSO Data)
{
    constitution = Data.constitution;
    hp = max_hp;
    strength = Data.strength;
    agility = Data.agility;
    wisdom = Data.wisdom;
    defence = Data.defence;
    armor = defence * 2;

    //skills = new();
    foreach (SkillSO skillSO in Data.skills)
    {
        AddSkillWithoutChecks(skillSO);
    }
}

public void AddSkill(SkillSO skillData)
{
    var skill = SkillDB.Instance.GetSkillByName(skillData.name);
    if (!skills.Contains(skill) && CheckSkillCount(skillData))
    {
        skills.Add(skill);
        CheckAddSkillGlobal(skill);
    }
}

public void AddSkillWithoutChecks(SkillSO skillData)
{
    var skill = SkillDB.Instance.GetSkillByName(skillData.name);
    skills.Add(skill);
}

public void AddSkill(Skill skill)
{
    if (!skills.Contains(skill) && CheckSkillCount(skill.skillData))
    {
        skills.Add(skill);
        CheckAddSkillGlobal(skill);
    }
}

public void AddSkill(string skillName)
{
    var skill = SkillDB.Instance.GetSkillByName(skillName);
    if (!skills.Contains(skill) && CheckSkillCount(skill.skillData))
    {
        skills.Add(skill);
        CheckAddSkillGlobal(skill);
    }
}

public bool CheckSkillCount(SkillSO data)
{
    int slots;
    bool isPassive = data.skill_target == SkillSO.SkillTarget.Passive;
    if (isPassive && data.isCurse)
    {
        slots = CurseSkillEmptySlots();
    }
    else if (isPassive)
    {
        slots = PassiveSkillEmptySlots();
    }
    else
    {

```

```

        slots = ActiveSkillEmptySlots();
    }
    return slots > 0;
}

public int ActiveSkillEmptySlots()
{
    int slots, cur_count;
    slots = 3 + agility / 3;
    if (slots > 7) slots = 7; //7 от статов + 1 от зелья = максимум 8
    try
    {
        if (SaveLoadController.runInfo.activePotion) slots++;
    }
    catch { };

    cur_count = 0;
    foreach (var skill in skills)
    {
        if (skill.skillData.skill_target != SkillSO.SkillTarget.Passive) cur_count++;
    }
    return slots - cur_count;
}

public int PassiveSkillEmptySlots()
{
    int slots, cur_count;
    slots = 2 + wisdom / 2;
    if (slots > 7) slots = 7; //7 от статов + 1 от зелья = максимум 8
    try
    {
        if (SaveLoadController.runInfo.passivePotion) slots++;
    }
    catch { };

    cur_count = 0;
    foreach (var skill in skills)
    {
        if (skill.skillData.skill_target == SkillSO.SkillTarget.Passive
            && !skill.skillData.isCurse) cur_count++;
    }
    return slots - cur_count;
}

public int CurseSkillEmptySlots()
{
    int slots, cur_count;
    slots = 8;

    cur_count = 0;
    foreach (var skill in skills)
    {
        if (skill.skillData.skill_target == SkillSO.SkillTarget.Passive
            && skill.skillData.isCurse) cur_count++;
    }
    return slots - cur_count;
}

public void CheckAddSkillGlobal(Skill skill)
{
    switch (skill.skillData.globalPassiveBuff)
    {
        case RunInfo.GlobalBuff.AmuletWind:
            var pool = SkillDB.Instance.GetPoolByName("WindSkills");
            var list = pool.activeSkillList;
            AddSkill(list[Random.Range(0, list.Count)]);
            return;
        case RunInfo.GlobalBuff.HeartDarkness:
            pool = SkillDB.Instance.GetPoolByName("DarkSkills");
            list = pool.activeSkillList;
            AddSkill(list[Random.Range(0, list.Count)]);
            return;
        case RunInfo.GlobalBuff.CursedHand:
            AddSkill(SkillDB.Instance.GetSkillByName("Purple Haze"));
            return;
    }
}

```

```

}

public void CastSkill(List<Fighter> targets, Skill skill, bool needCooldown = true)
{
    bool isDoubleNextAttack = false;
    List<Fighter> doubleTargets = targets;

    //Проверка на баффы у кастующего
    for (int i = 0; i < buffs.Count; i++)
    {
        switch (buffs[i])
        {
            case Buff.DoubleNextAttack:
                if (skill.skillData.skill_type != SkillSO.SkillType.Attack) continue;
                isDoubleNextAttack = true;
                doubleTargets = Fight.ChooseTarget(skill, this, Fight.RandomEnemy(this));
                buffs.RemoveAt(i);
                i--;
                break;

            case Buff.BestialInstinct:
                if (skill.skillData.skill_type == SkillSO.SkillType.Attack)
                {
                    if (buffs.Contains(Buff.BestialInstinctBuff))
                        buffs.Remove(Buff.BestialInstinctBuff);
                    continue;
                }
                if (!buffs.Contains(Buff.BestialInstinctBuff))
                    buffs.Add(Buff.BestialInstinctBuff);
                break;

            case Buff.EchoForest:
                if (skill.skillData.skill_type == SkillSO.SkillType.Attack) continue;
                armor += GetPassiveSkill(Buff.EchoForest).calc(new List<Fighter> { this })[0];
                break;

            case Buff.ProvocationCaster:
                List<Fighter> allies = new List<Fighter>(Fight.PlayerTeam);
                if (Fight.IsEnemy(this, allies[0]))
                    allies = new List<Fighter>(Fight.EnemyTeam);
                foreach (var chara in allies)
                {
                    if (chara.buffs.Contains(Buff.Provocation))
                        chara.buffs.Remove(Buff.Provocation);
                }
                buffs.Remove(Buff.ProvocationCaster);
                break;

            case Buff.DissolvingShadows:
                buffs.Remove(Buff.DissolvingShadows);
                break;

            case Buff.Accompaniment:
                buffs.Remove(Buff.Accompaniment);
                isDoubleNextAttack = true;
                SacrificeHP(max_hp);
                break;
        }
    }
    skill.Cast(this, targets);
    var cooldown = skill.skillData.cooldown;
    if (cooldown != 0 && needCooldown)
        cooldowns.Add(skill, cooldown);

    //Проверка на баффы после применения заклинания
    if (buffs.Contains(Buff.PainSilencing))
    {
        SacrificeHP((int)((max_hp + bonus_hp) * 0.03f));
    }

    if (isDoubleNextAttack && !isDead)
    {
        Fight.additionalCastSkills.Add(new(doubleTargets, false, skill));
    }
}

public Sprite Portrait

```



```

{
    get
    {
        if (this is PlayableCharacter) return (this as PlayableCharacter).Portrait;
        return Data.portrait_monster;
    }
    private set { }
}

public void PrepareToFight()
{
    skillsBattle = new(Skills);
}

public void Spawn()
{
    if (isSpawn) return;

    PrepareToFight();

    //Все уже появившиеся на поле
    var allSpawned = new List<Fighter>();
    foreach (Fighter character in Fight.AllCharacter)
        if (character.isSpawn) allSpawned.Add(character);

    //Свои абилки
    foreach (Skill skill in skillsBattle)
    {
        //skill.battlecry?.Invoke();
        skill.passive?.Invoke(this, allSpawned);
    }

    //Действующие на него абилки
    foreach (Fighter character in allSpawned)
    {
        if (character != null && character != this)
            foreach (Skill skill in character.skillsBattle)
            {
                //passive
                skill.passive?.Invoke(character, new List<Fighter> { this });
            }
    }

    //Баффы от пассивок
    foreach (var skill in skillsBattle)
    {
        var skillData = skill.skillData;
        if (skillData.passiveBuff != Buff.None)
        {
            buffs.Add(skillData.passiveBuff);
        }
    }

    if (Fight.EnemyTeam.Contains(this))
        MakeIntention();
    isSpawn = true;
}

public void MakeIntention()
{
    prevIntension = Intension;

    //Количество скиллов без пассивок и перезаряжаемых скиллов
    //int skillCount = 0;
    List<Skill> availableSkills = new();
    foreach (Skill skill in skillsBattle)
    {
        if (skill.skillData.skill_target != SkillSO.SkillTarget.Passive
            && !cooldowns.Keys.Contains(skill))
            availableSkills.Add(skill);
    }
    if (buffs.Contains(Buff.NoAttack))
        availableSkills.Remove(SkillDB.Instance.GetSkillByName("Basic Attack"));
    availableSkills.Remove(prevIntension);
}

```

```

//bool reroll;
var count = availableSkills.Count;
if (count > 0)
{
    Intension = availableSkills[Random.Range(0, count)];
}
else
{
    if (cooldowns.Keys.Contains(prevIntension))
    {
        Intension = null;
        //Fight.skipTurn = true;
    }
}
}

public void Death()
{
    if(this == SaveLoadController.runInfo.PlayerTeam[0])
    {
        //Проверка на бессмертие (работает, но не уверен в плане баланса, отключено):
        /*
        int chance = (strength + bonus_strength) * Fight.procentPerOneCharacteristic;
        if (chance > Fight.limitProcent) chance = Fight.limitProcent;
        int res = Random.Range(0, 100);
        if(res < chance)
        {
            hp = 1;
            return;
        }*/
    }

    //Баффы перед смертью
    foreach (var buff in buffs)
    {
        switch (buff)
        {
            case Buff.AngelicGrace:
                hp = max_hp + bonus_hp;
                buffs.Remove(Buff.AngelicGrace);
                effectStacks.Remove(Effect.AngelicGrace);
                var grace = SkillDB.Instance.GetSkillByName("Angelic Grace");
                skills.Remove(grace);
                skillsBattle.Remove(grace);
                return;
        }
    }

    isDead = true;

    //Баффы при смерти
    foreach (var buff in buffs)
    {
        switch (buff)
        {
            case Buff.ProvocationCaster:
                List<Fighter> allies = new List<Fighter>(Fight.PlayerTeam);
                if (Fight.IsEnemy(this, allies[0]))
                    allies = new List<Fighter>(Fight.EnemyTeam);
                foreach (var chara in allies)
                {
                    if (chara.buffs.Contains(Buff.Provocation))
                        chara.buffs.Remove(Buff.Provocation);
                }
                buffs.Remove(Buff.ProvocationCaster);
                break;
        }
    }

    foreach (Skill skill in skillsBattle)
    {
        skill.death?.Invoke(new List<Fighter> { this });
        skill.reverse?.Invoke(this, Fight.AllCharacter);
    }
}

```

```

public Skill GetPassiveSkill(Buff buff)
{
    foreach(var skill in skillsBattle)
    {
        if (skill.skillData.passiveBuff == buff)
            return skill;
    }
    return null;
}

public void TakeDmg(Fighter takeFrom, int dmg, SkillSO.SkillElement element, int
counterRecursion = 0)
{
    if (counterRecursion > 10000) return;
    //Buffs for DmgCaster, not include poison and curse
    if (takeFrom != null)
        foreach (var buff in takeFrom.buffs)
        {
            switch (buff)
            {
                case Buff.OldFightersPlate:
                    break;
            }
        }

    //Buffs
    foreach (var buff in buffs)
    {
        switch (buff)
        {
            case Buff.OldFightersPlate:
                dmg -= GetPassiveSkill(buff).calc(new List<Fighter> { this })[0];
                break;
            case Buff.Provocation:
                dmg = 0;
                break;
            case Buff.DissolvingShadows:
                if (element != SkillSO.SkillElement.Light)
                    dmg = 0;
                break;
            case Buff.EchoPain:
                switch(Random.Range(0,3))
                {
                    case 0:
                        bonus_strength++;
                        break;
                    case 1:
                        bonus_agility++;
                        break;
                    case 2:
                        bonus_wisdow++;
                        break;
                }
                break;
            case Buff.PainSilencing:
                if(dmg > 1)
                    dmg = 1;
                break;
            case Buff.AngelicPower:
                int heal = dmg / 3;
                if (heal < 1) heal = 1;
                TakeHeal(this, heal, counterRecursion);
                var enemy = Fight.RandomEnemy(this);
                enemy.TakeDmg(takeFrom, heal * 2, SkillSO.SkillElement.Light,
counterRecursion);
                break;
        }
    }

    if (dmg < 0) return;
    if(armor > 0)
    {
        var armor_cur = armor;
        armor -= dmg;
    }
}

```

```

        dmg -= armor_cur;
        if (armor < 0) armor = 0;
    }
    if (dmg >= 0)
    {
        hp -= dmg;
        if (hp <= 0)
        {
            hp = 0;
            Death();
        }
    }
}

public void SacrificeHP(int _hp)
{
    int dmg = _hp;
    if (dmg <= 0) dmg = 1;

    if (dmg >= 0)
    {
        hp -= dmg;
        if (hp <= 0)
        {
            hp = 0;
            Death();
        }
    }
}

public int TakeHeal(Fighter takeFrom, int heal, int counterRecursion = 0)
{
    if (counterRecursion > 10000) return 0;
    int excess = 0;
    if (heal >= 0)
    {
        excess = heal - (max_hp + bonus_hp - hp);
        hp += heal;
        if (hp > max_hp + bonus_hp)
        {
            hp = max_hp + bonus_hp;
        }
    }
    if (excess < 0) excess = 0;

    //Buffs for HealCaster
    if (takeFrom != null)
        foreach (var buff in takeFrom.buffs)
        {
            switch (buff)
            {
                case Buff.VengefulLight:
                    var enemy = Fight.RandomEnemy(takeFrom);
                    enemy.TakeDmg(takeFrom, excess, SkillSO.SkillElement.Light,
counterRecursion);
                    break;
            }
        }

    return excess;
}

public void FullHeal()
{
    hp = max_hp + bonus_hp;
}

internal Skill BuffToSkill(Buff buff)
{
    string name = "";
    switch (buff)
    {
        case Buff.DoubleNextAttack:
            name = "Waiting";
            break;
    }
}

```

```

        case Buff.OldFightersPlate:
            name = "Old Fighter's Chest";
            break;
        case Buff.BestialInstinctBuff:
            name = "Bestial Instinct";
            break;
        case Buff.Corpseless:
            name = "Corpseless";
            break;
        case Buff.CurseDestruction:
            name = "Curse of Destruction";
            break;
        case Buff.CurseVictim:
            name = "Curse of Victim";
            break;
        case Buff.WeightMemories:
            name = "The Weight of Memories";
            break;
        case Buff.CallPack:
            name = "Call of the Pack";
            break;
        case Buff.EchoPain:
            name = "Echo of Pain";
            break;
        case Buff.EchoForest:
            name = "Echo of Forest";
            break;
        case Buff.Provocation:
            name = "Provocation";
            break;
        case Buff.Poison:
            name = "Poison";
            break;
        case Buff.DissolvingShadows:
            name = "Dissolving in the Shadows";
            break;
        case Buff.PainSilencing:
            name = "Pain Silencing";
            break;
        case Buff.SacrificialChant:
            name = "Sacrificial Chant";
            break;
        case Buff.Accompaniment:
            name = "Accompaniment";
            break;
        case Buff.NoAttack:
            name = "No Attack";
            break;
        case Buff.VengefullLight:
            name = "Vengeful Light";
            break;
        case Buff.AngelicGrace:
            name = "Angelic Grace";
            break;
        case Buff.AngelicPower:
            name = "Angelic Power";
            break;
        case Buff.ScarBetrayal:
            name = "Scar of Betrayal";
            break;
    }
    if (name == "") return null;
    return SkillDB.Instance.GetSkillByName(name);
}

public enum Effect
{
    AngelicGrace,
}

public enum Buff
{
    None,
    DoubleNextAttack,
    OldFightersPlate,

```

```

QuickRebuff,
BestialInstinct,
BestialInstinctBuff, // накладывается когда применяешь не атакующий навык
QuietBlessing,
ScreamIntoVoid,
Corpseless,
CursedHand,
TouchingMystery,
CurseDestruction,
WeightMemories,
CallPack,
EchoHope,
EchoPain,
EchoForest,
Provocation,
ProvocationCaster,
Poison,
DissolvingShadows,
PainSilencing,
SacrificialChant,
CurseVictim,
Accompaniment,
NoAttack,
WanderingMusician,
VengefulLight,
AngelicGrace,
AngelicPower,
AngelicGuardian,
ScarBetrayal,
    }
}

```